Digital Banking

Engineering Companion

Volume II of the Digital Banking Engineering Series

John Whitton

Version 1.1.0 · 2026-07-17

Scope. This vendor-neutral companion explains how to implement, test, deploy, and operate the architecture defined by Volume I. It is not production certification, procurement approval, legal advice, or a runnable implementation. Zelle remains a public case study, not an implementation claim. Code evidence is limited to the exact commit and status stated in this volume.

Reader guide

Purpose. Volume I defines the architecture; this companion turns its durable-operation, ledger, finality, and authority contracts into implementable boundaries. It is a reference for engineers, security reviewers, operators, and technical decision makers. It is deliberately profile-neutral: a named product is a candidate or an example, never an unqualified endorsement.

Authority and evidence. Architecture authority flows from the series specifications, glossary, and accepted ADRs. Implementation evidence flows upward from source and tests and may show only whether one implementation currently realizes a contract. These directions must not be reversed. The implementation repository was inspected read-only at commit `e921fcb1877b46a6881437f46b1a6ebfa115ae58`. Generated output and uncommitted files were excluded. No claim about that commit is a production-certification claim.

Table 1. Implementation status vocabulary used throughout this volume.

Status	Meaning	Reader rule
implemented	Source and tests realize the bounded behavior at the pinned commit.	Do not infer deployment or production readiness.
partially implemented	Some required behavior exists, with material execution or assurance gaps.	Read the limitation before relying on the claim.
interface only	A port or type exists without a conforming runtime adapter.	Treat as a design seam, not functionality.
planned	The contract is specified but absent from the pinned source.	Requires implementation and evidence.
intentionally excluded	Scope was consciously deferred or rejected for the current phase.	Re-open only through governance.
unresolved	Evidence or a design decision is insufficient.	Block dependent readiness claims.

How to read the diagrams. Solid arrows show control or dependency direction; dashed amber arrows show evidence. Red nodes are authority or ambiguity boundaries. Every figure has an editable TikZ source, a stable label, reading order, and evidence classification. Tables preserve *not applicable*, *not evaluated*, *unknown*, *unsupported*, and *failed gate* as distinct values; a blank cell is never a favorable result.

Non-goals. This volume does not prescribe a bank's risk appetite, declare a chain or provider production-ready, replace threat modeling or legal analysis, or authorize issuance. It does not make Zelle an implementation dependency. It does not start Volume III.

Evidence boundary. The implementation-status ledger under `evidence/` is the machine-readable claim boundary. Reader-facing citations support technical facts; the annotated bibliography records why each source was used and what it does not establish.

Code-reading rule. Architectural contracts and public interfaces appear only as short, exact excerpts from the pinned implementation, with repository-relative paths. Workflow examples are labeled illustrative pseudocode and optimize for explanation rather than source fidelity. The pinned signer result type is `SigningDecision`; no `SigningResult` type exists at that commit. Volume III remains the future home for complete executable examples.

Contents

List of Figures

List of Tables

Part I.
Part I — Engineering Foundations

1. Engineering foundations

Why this chapter. Blockchain settlement becomes operable only after money, identity, state, retry, and evidence have durable owners. This chapter establishes that engineering spine before the technology-specific chapters add implementation choices.

1.1. The ledger and durable identity

The authoritative internal ledger is the institution's system of record for balances, reservations, postings, fees, and corrections. A blockchain record is external settlement evidence; it does not replace internal accounting. Represent money as an integer minor-unit quantity plus an immutable asset/version identifier and explicit scale. Reject floating-point money, implicit rounding, and cross-asset arithmetic. Preserve the submitted amount, assessed fees, reserved amount, settled amount, and any correction as separate fields.

The durable identity chain is: payment intent → operation → attempt → observation. A transition has its own immutable identifier and expected prior version. Finality observations identify the chain, network, transaction or signature, block or slot lineage, policy version, observer, and observation time. Evidence objects identify their producer, digest, schema version, and retention class. Correlation IDs aid diagnosis but are not business identities.

Invariant. Exactly-once applies to business effects, not message delivery. At-least-once delivery is safe only when each consumer applies a durable deduplication key in the same transaction as its business effect.

1.2. State, commands, and retries

An operation is a durable state machine, not an HTTP request. Admission validates syntax, participant scope, asset, limits, and idempotency before returning an operation identity. A worker advances one authorized transition at a time using optimistic version checks or a narrowly scoped row lock. Transition facts are append-only; the current-state projection is rebuildable. Terminal failure never erases a prior reservation, signature, submission, or observation.

Canonicalize a command before hashing it: version the schema; normalize Unicode and identifiers; serialize fields in a fixed order; preserve numeric scale; distinguish absent from null; bind the participant, operation type, asset, amount, destination, policy context, and idempotency key. Reject reuse of a key with a different canonical digest. The response replay must refer to the original operation and never re-run admission effects.

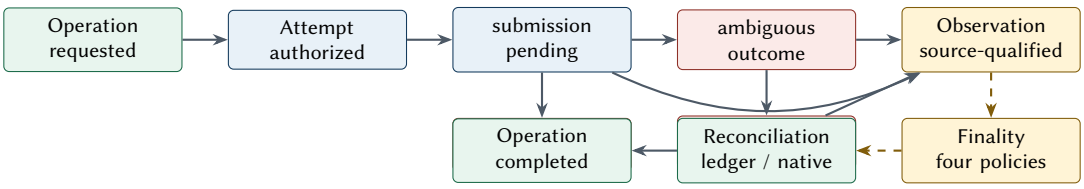

Figure 1. Non-exhaustive durable Operation, Attempt, Observation, Finality, and Reconciliation relationships. *Reading order:* this non-exhaustive view shows an Operation owning linked Attempts; submission may proceed directly to Observation or become ambiguous and require inquiry/manual review. Source-qualified observations feed four independent Finality judgments, then Reconciliation compares native evidence with ledger truth. Earlier policy rejection branches are omitted for legibility. Domain states are implemented; runtime progression after acceptance is planned.

A retry is an authorized state transition. It requires a retry class, owner, deadline, attempt budget, and proof that the preceding outcome is known or safely recoverable. A timeout after submission is *ambiguous*: inquire by signed transaction identity and independent observation before considering rebroadcast. If a changed nonce, fee, blockhash, or message is needed, create a linked new attempt; do not mutate signed evidence.

Table 2. Failure classes and ownership.

Class	Example	Required action	Owner
pre-effect retryable	transient simulation or provider refusal	Retry under bounded policy; preserve attempt evidence.	adapter/service owner
ambiguous	timeout after submission	Inquire and observe; never blind-retry a new business effect.	recovery worker and on-call
chain conflict	EVM nonce replacement or Solana expiry	Link a new native attempt under explicit authorization.	chain adapter owner
business rejection	participant, funds, limit, or policy failure	Record final reason without exposing sensitive detail.	control-plane/domain owner
reconciliation break	ledger and native evidence disagree	Freeze affected progression, open a case, preserve evidence.	finance operations
security event	unauthorized signing request or key anomaly	Deny, isolate, rotate or revoke, and invoke incident response.	security and signer owners

1.3. Transactions, messaging, and correction

Use one database transaction to commit an accepted operation, its transition, and an outbox record. A separate publisher claims outbox rows with leases, emits at least once, and records delivery metadata. Consumers write an inbox/deduplication record with their business effect. A broker acknowledgement is transport evidence only. Recovery scans for expired leases, stalled states, and missing expected evidence; it does not infer success from elapsed time.

Reconciliation compares the authoritative ledger, operation/attempt records, signer evidence, provider responses, and independent native observations. Differences become named breaks with owner, age, materiality, and disposition. Corrections and compensations are new authorized entries linked to the original; immutable history is not edited. A compensation reverses a business effect where reversal is valid. It is not proof that an irreversible external effect was undone.

Table 3. Four independent finality policies.

Finality	Evidence	Engineering consequence
blockchain finality	approved native-chain evidence for the exact transaction and state effect	Re-evaluate across reorgs, forks, commitment levels, or L2 settlement dependencies.
legal settlement finality	applicable law, rules, agreements, operating procedures, and legal analysis	Never infer solely from an RPC response, confirmation count, or business label.
customer-visible finality	the status and remedy promise exposed to a participant or customer	Bind wording, compensation, support, liquidity, and loss allocation to an approved policy.
accounting finality	accepted postings and treatment under ledger policy	Further correction requires a governed reversal, adjustment, or later-period entry; external reconciliation is evidence, not the definition.

Network confidence and residual economic exposure inform blockchain and customer-visible policy; they are not substitute finality categories. One finality may advance before another only through an explicit policy that owns exposure, liquidity, remedy, correction, and reconciliation.

Authentication, business authorization, wallet action, signer policy, and on-chain authority are five distinct controls. A success in one does not imply another. Failure ownership is similarly explicit: every transition, queue, provider call, alert, and reconciliation break has one accountable service/team and a documented escalation path.

Implementation reference. Domain identities and operation lifecycle — `digital-banking@ e921fcb1877b46a6881437f46b1a6ebfa115ae58`; module `domain`; package `io.github.johnwhitton.digitalbanking.domain.operation`; `domain/src/main/java/io/github/johnwhitton/digitalbanking/domain/operation`; status: **implemented**.

Implementation notes. The plain-Java domain implements typed identities, exact amounts, lifecycle states, and transition guards. Application acceptance and PostgreSQL persistence are implemented, but exact TokenQuantity types are not a ledger/accounting implementation: no authoritative double-entry ledger or accounting module exists at the pinned commit. Outbox delivery is partial, and runtime workers, signing, chain execution, finality driving, and reconciliation are planned. Durable acceptance is not proof of mint, burn, external effect, or settlement.

Continue the thread. Volume I: *Internal ledger*, *Durable payment state machine*, and *Four independent finalities*. Volume III planned chapter: domain contracts and control plane. ADRs: ADR-001 and ADR-002. Engineering modules: `shared/engineering/README.md`. Implementation: `domain/src/main/java` and `application/src/main/java`.

Part II.
Part II — Java and Spring Control Plane

2. Java and Spring control plane

Why this chapter. The architecture needs a control plane that accepts business intent durably without importing transport, database, or chain-provider authority into the domain. Java module boundaries make that separation executable and reviewable.

2.1. Module contracts and dependency direction

Use a Maven reactor to make dependency direction executable: domain contains plain Java value objects, aggregates, and invariants; application contains use cases and ports; adapters implement database, messaging, signer, and chain ports; control-plane composes Spring Boot HTTP/security/operations concerns. Parent dependency management centralizes versions, but each module declares only direct dependencies. The domain must not import Spring, JDBC, JSON, provider SDK, or generated API types. Maven's reactor orders modules; it does not enforce architecture by itself, so add dependency-rule tests. (Apache Software Foundation 2026; Broadcom 2026b)

Implementation reference. Stable operation identity — `digital-banking@ e921fcb1877b46a6881437f46b1a6ebfa115ae58`; module `domain`; package `io.github.johnwhitton.digitalbanking.domain.operation`; `domain/src/main/java/io/github/johnwhitton/digitalbanking/domain/operation/OperationId.java`; status: **implemented**.

Exact pinned excerpt: `OperationId` identity and canonical parser entry

`digital-banking@e921fcb1877b46a6881437f46b1a6ebfa115ae58 · domain/src/main/java/io/github/johnwhitton/digitalbanking/domain/operation/OperationId.java`

```
/** Stable identity issued before an operation begins. */
public record OperationId(UUID value) {

    public OperationId {
        Objects.requireNonNull(value, "value");
    }

    public static OperationId from(String canonical) {
        return new OperationId(parseCanonical(canonical));
    }
```

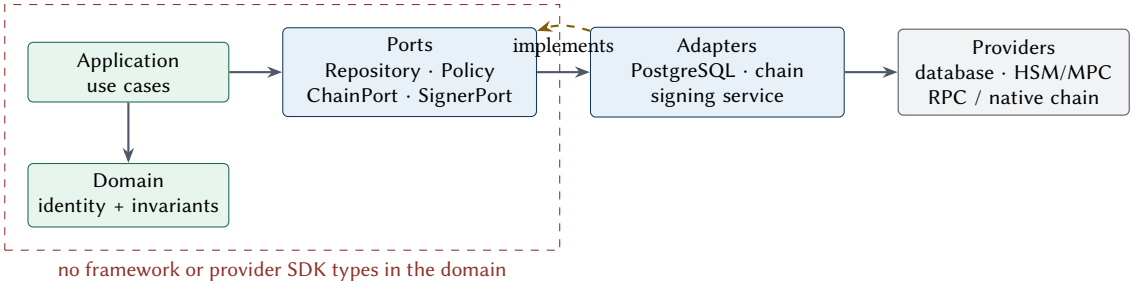

Figure 2. Application, Port, Adapter, and Provider boundaries.
Reading order: runtime intent moves from the Application through a semantic Port to an Adapter and then a Provider. Compile-time dependencies still point inward: adapters implement application ports, and the domain imports nothing outer. Evidence status: domain, application, PostgreSQL, and Spring modules are implemented; ChainPort and SignerPort have no runtime adapter at the pinned commit.

The implementation uses a UUID value object but accepts only canonical UUID text when parsing. The architectural point is stable pre-execution identity; the concrete UUID choice remains implementation evidence.

Implementation reference. Atomic acceptance repository — digital-banking@ e921fcb1877b46a6881437f46b1a6ebfa115ae58; module application; package io.github.johnwhitton.digitalbanking.application.port; application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/OperationRepository.java; status: **implemented**.

Exact pinned excerpt: `OperationRepository.accept` contract

digital-banking@e921fcb1877b46a6881437f46b1a6ebfa115ae58 · application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/OperationRepository.java

```
    OperationAcceptance accept(
            IdempotencyScope scope,
            IdempotencyKey key,
            CanonicalCommandMetadata canonicalCommand,
            Supplier<TokenOperation> operationFactory);
```

The acceptance call binds scope, key, canonical command, and operation creation at one repository boundary. The PostgreSQL adapter supplies durability; the interface alone does not.

Implementation reference. Attempt read projection — digital-banking@ e921fcb1877b46a6881437f46b1a6ebfa115ae58; module application; package io.github.johnwhitton.digitalbanking.application.port; application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/AttemptRepository.java; status: **interface only**.

Exact pinned excerpt: `AttemptRepository` reads

digital-banking@e921fcb1877b46a6881437f46b1a6ebfa115ae58 · application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/AttemptRepository.java

```
public interface AttemptRepository {

    Optional<OperationAttempt> find(OperationId operationId, AttemptId attemptId);

    List<OperationAttempt> findByOperation(OperationId operationId);
}
```

Attempt creation deliberately stays behind the guarded aggregate save; this interface is a read-side projection rather than a second writer with weaker transition rules.

Define application commands with domain types, not transport DTOs. Adapters translate at the edge and contain vendor exceptions, wire encodings, pagination, clocks, and retry metadata. Ports are narrow business capabilities: save an operation with expected version, reserve a nonce, evaluate a contextual signing request, submit immutable signed bytes, or obtain a native observation. Avoid a generic provider facade that erases native semantics.

2.2. HTTP, identity, authorization, and validation

Generate or validate an OpenAPI contract in CI and publish explicit error schemas. On creation, return `202 Accepted` with a durable operation URL only after the operation and outbox record commit. Use `202 Accepted` for exact idempotent replay, `409` for key/digest conflict, and bounded problem details for validation and authorization failures. Never let a controller construct provider payloads. (OpenAPI Initiative 2026)

Implementation reference. Mint acceptance API — digital-banking@ e921fcb1877b46a6881437f46b1a6ebfa115ae58; module control-plane; OpenAPI 3.1 contract; control-plane/src/main/resources/static/openapi/token-operations-v1.yaml; status: **partially implemented**.

Exact pinned excerpt: mint acceptance endpoint

digital-banking@e921fcb1877b46a6881437f46b1a6ebfa115ae58 · control-plane/src/main/resources/static/openapi/token-operations-v1.yaml

```
/v1/token-operations/mints:
  post:
    operationId: acceptMintOperation
    summary: Durably accept a mint operation request
    description: >-
      A replay with the same participant scope, key, and canonical request
      returns HTTP 202 and the same operation without creating another event.
    security:
      - identityAdapter: []
    x-required-authority: token:mint
```

This endpoint accepts work; it does not claim that signing, submission, minting, finality, or reconciliation has occurred.

Spring Security authenticates workload or participant identity, but the application authorizes the requested participant, asset, role, operation, and limit. Decode identity through an environment-specific component; deny when identity or participant scope is absent. Apply both endpoint-level and use-case authorization, because alternate transports and internal calls must not bypass the rule. Avoid participant enumeration and sensitive policy detail in responses or logs. (Broadcom 2026d)

Perform cheap structural validation at the edge, then authoritative validation in the domain and application transaction. Validate exact amount scale, asset status, destination form, participant scope, key uniqueness, and policy version. Database constraints remain the last line for uniqueness and referential integrity; they are not a substitute for a domain error.

2.3. PostgreSQL, Flyway, and transaction boundaries

PostgreSQL is the durable coordination store. Use explicit unique constraints for request and attempt identities, immutable transition rows, foreign keys for evidence lineage, and a version column for optimistic concurrency. Choose an isolation level from measured anomalies; at the default read-committed level, each statement sees a snapshot, so a read-then-write sequence needs a conditional update, lock, or constraint. Retry a serialization failure at the whole application-command boundary, not midway through effects. (PostgreSQL Global Development Group 2026)

Flyway migrations are append-only release artifacts. CI applies them to an empty database and to a supported previous release, then runs contract tests. Use expand/migrate/contract changes: deploy backward-compatible schema, dual-read/write only when necessary, backfill with checkpoints, switch readers, and remove old structure later. Rollback normally means application rollback with forward database repair; down migrations are unsafe once data meaning changed. (Redgate 2026)

One Spring transaction owns operation state, ledger effect, and outbox insert. External RPC, signing, or broker calls never occur while a row lock or database transaction is open. A worker claims durable work, commits the lease, performs bounded external work, then commits the observed result using expected state/version. Spring transaction propagation should be explicit and tested; proxy annotations do not rescue a wrongly drawn business boundary. (Broadcom 2026c)

2.4. Messaging, configuration, and operations

An outbox publisher uses lease owner, lease expiry, attempt count, next-at time, and immutable payload digest. Ordering is per business aggregate where required, not global. Consumers use an inbox key scoped to producer and message identity. Poison messages enter a governed quarantine with replay tooling; a dead-letter queue is not resolution.

Bind typed configuration with validation and safe defaults. Separate non-secret profile data from secret references. Refuse startup for unknown chain IDs, missing policy versions, writable mock adapters in production, or observation sourced from the same failure domain when independence is required. Record effective non-secret configuration and artifact digest for each deployment.

OpenTelemetry traces connect admission, worker attempts, signer requests, submission, observation, and reconciliation without carrying private payloads or credentials. Metrics are low-cardinality: state age, queue depth, lease expiry, policy denials, provider latency/errors, ambiguous attempts, reconciliation breaks, and finality lag. Logs are structured diagnostic events; the durable audit record is a separately controlled evidence stream. (OpenTelemetry Authors 2026)

Spring Actuator liveness answers whether the process should restart. Readiness answers whether it can safely take new work, considering database and required policy/configuration but avoiding cascading restarts for every provider incident. Expose administrative endpoints on a restricted management plane. (Broadcom 2026a)

Test the domain without Spring; application use cases against fakes and contract suites; JDBC and migrations with Testcontainers; HTTP/security with real filters; and production wiring with a minimal environment. A deployment gate includes schema compatibility, rollback rehearsal, readiness behavior, alert coverage, and a canary that performs no uncontrolled value movement.

Implementation reference. Spring API and participant security — `digital-banking@ e921fcb1877b46a6881437f46b1a6ebfa115ae58`; module `control-plane`; package `io.github.johnwhitton.digitalbanking.controlplane`; `control-plane/src/main/java/io/github/johnwhitton/digitalbanking/controlplane`; status: **partially implemented**.

Implementation notes. The four-module direction, request path, JDBC/Flyway persistence, and test fixtures are implemented. No identity adapter/authentication mechanism is configured; endpoints deny by default; the asset catalog returns empty; tests inject fixture principals. Endpoint authorities exist, but request acceptance does not yet invoke application role, limit, or policy authorization. Worker/messaging, signer/chain adapters, production identity/catalog, telemetry, and deployment remain partial or planned.

Continue the thread. Volume I: *Java architecture and language boundaries* and *Consistency: narrow atomicity, end-to-end recoverability.* Volume III planned chapter: domain contracts, control plane, and messaging. ADRs: ADR-004 and ADR-005. Engineering modules: `shared/engineering/sdk/java/README.md`. Implementation: `control-plane/src/main/java`, `application/src/main/java`, and `adapters/persistence-postgres/src/main/java`.

Part III.
Part III — Wallet, Custody, and Signing

3. Wallet, custody, and signing

Why this chapter. A protected key answers who can invoke cryptography, not whether a proposed transaction is permitted. Wallet roles and contextual signing therefore form an authorization system with explicit business, policy, provider, and native-chain evidence.

Security boundary. An HSM prevents private-key extraction, but does not by itself prevent a compromised application from requesting a malicious signature. The signing boundary must independently authorize what the payload means.

3.1. Roles and authority boundaries

Keep customer/participant identity, treasury funding, fee payment, mint authority, burn authority, contract administration, emergency pause, upgrade, observation, and reconciliation roles separate. One physical provider may implement several roles, but the registry, policies, keys, credentials, approvals, and audit identities remain distinguishable.

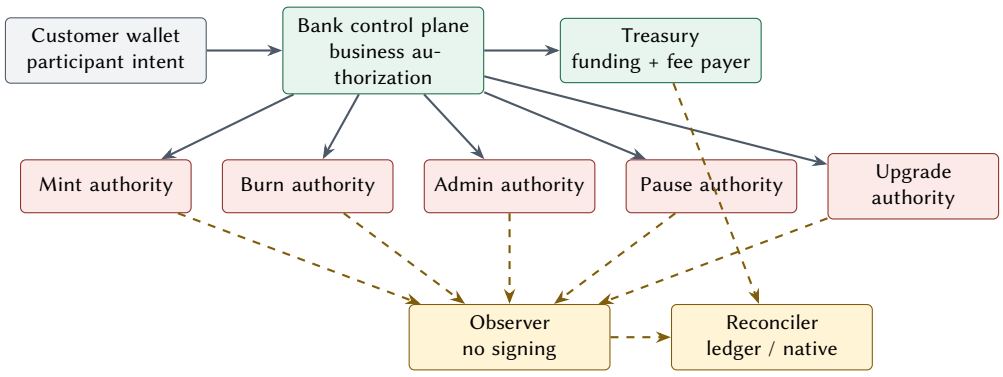

Figure 3. Customer wallet, Bank, Treasury, transactional authorities, governance, Observer, and Reconciler.
Reading order: Customer wallet identity provides participant intent to the Bank control plane; Treasury owns funding and fee posture; Mint, Burn, Admin, Pause, and Upgrade authorities remain distinct even if one provider hosts several keys. The Observer cannot sign, and the Reconciler compares independent native evidence with ledger truth. The exact physical wallet/custody model remains a profile decision.

Table 4. Signer and wallet roles.

Role	Permitted context	Separation rule
fee payer	Pay network fees for an approved attempt.	Cannot grant mint/burn or upgrade authority.
mint authority	Execute a bounded asset issuance for an approved operation.	Separate key and policy from burn and governance authorities.
burn authority	Execute a bounded asset retirement for an approved operation.	Separate key and policy from mint and governance authorities.
contract/program administrator	Change bounded administrative configuration under governance.	Offline or quorum path; never the routine transaction worker.
emergency pause authority	Pause an affected contract or program under incident governance.	Distinct credential, approval, and audit identity.
upgrade authority	Approve and execute a reviewed implementation upgrade.	Distinct quorum/timelock path with rollback evidence.
signing service	Decode, reconstruct, authorize, invoke protected key, verify.	No business admission and no unilateral on-chain grant.
observer	Gather independent native evidence.	Has no signing capability.
reconciler	Compare ledger and external evidence and open breaks.	Cannot rewrite history or silently retry.

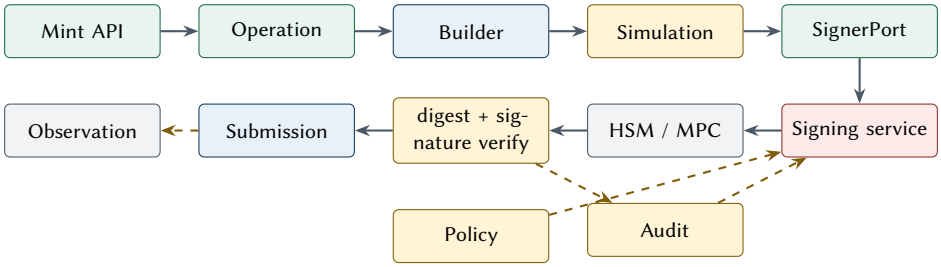

Figure 4. Mint API through contextual signing, protected cryptography, Submission, and independent Observation. *Reading order:* the Mint API creates a durable Operation; the Builder and Simulation bind native evidence; SignerPort passes that context to a Signing service that independently decodes and applies Policy before an HSM signs. The returned digest and signature are verified and written to Audit before Submission. Observation remains independent. Evidence status at the pinned commit: durable API/operation acceptance is implemented; SignerPort and ChainPort are interface only; signing service, HSM adapter, Submission, and Observation runtimes are absent.

Exact pinned excerpt: `SignerPort` operation

`digital-banking@e921fcb1877b46a6881437f46b1a6ebfa115ae58 · application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/SignerPort.java`

```
public interface SignerPort {

    SigningDecision sign(SigningRequest request);

    record SigningRequest(
            String signerRequestIdentity,
            OperationId operationId,
            AttemptId attemptId,
            OperationKind kind,
            String chainIdentity,
            String routeVersion,
```

The request continues in source with asset, exact quantity, authorities, destination, native constraint and payload digests, fee and allowlist bounds, lifetime, policy version, and approval and simulation evidence. Those fields make the port contextual rather than a byte-signing oracle.

Exact pinned excerpt: pinned signing result type

digital-banking@e921fcb1877b46a6881437f46b1a6ebfa115ae58 · application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/SignerPort.java

```
    record SigningDecision(
            Decision decision,
            String providerRequestIdentity,
            String keyReference,
            byte[] signedPayload,
            String digestReference,
            String reason,
            String signerPolicyVersion,
            EvidenceRef authorizationEvidence) {
```

The exact result type is `SigningDecision`, not `SigningResult`. Its constructor validation and defensive byte-array copies remain authoritative in the pinned source.

3.2. Contextual signing contract

The application submits a versioned request containing operation and attempt identities, chain and network, asset/action, expected contract or program, participant/tenant, amount, destination, fee/nonce or blockhash constraints, payload digest, policy version, approval references, and expiry. The dedicated signing service fetches authoritative key and policy data, independently decodes the complete native payload, reconstructs the digest, and compares every material field to the request. It rejects extra calls, writable accounts, delegates, authorities, instructions, value transfers, chain IDs, or fee conditions outside policy.

There is no generic byte-array signing method on the business port. Such an API becomes an arbitrary signing oracle. Provider-specific low-level signing is private to a native adapter after the policy decision. The response binds request digest, key version, native algorithm, signature, provider request identity, policy decision, and audit reference. Verify the returned signature and recovered/derived public identity before releasing signed bytes.

Before the first provider call, durably bind the signer request identity, canonical request hash, and key version. For that binding, exact replay returns the original signature or signed payload already recorded; a changed input is a conflict, not a new signature under the same identity. An unknown provider outcome enters inquiry or quarantine and never authorizes re-signing. The application stores signed transaction bytes under encryption and controlled access with digest, integrity, retention, public-key, policy, and evidence metadata—never private-key material, seed phrases, shares, or export blobs. Raw unsigned or signed payloads are never written to logs.

3.3. Key registry, lifecycle, and recovery

The key registry is an application control record, not secret storage. Each version records a stable logical key ID, provider locator, public key/address, algorithm and curve, environment, allowed roles/networks/assets/actions, state, activation window, policy version, and evidence links. Provider aliases are resolved to immutable versions for an attempt. Disable unknown or unverified versions.

Rotation spans application registry, signing policy, provider key, on-chain authority, observer, and reconciliation. Generate in the protected boundary; register and verify the public identity; grant narrow on-chain authority; run non-value or bounded tests; cut over; drain in-flight attempts; revoke old authority; then disable, archive, or destroy according to retention and recovery policy. Emergency revocation favors containment over availability.

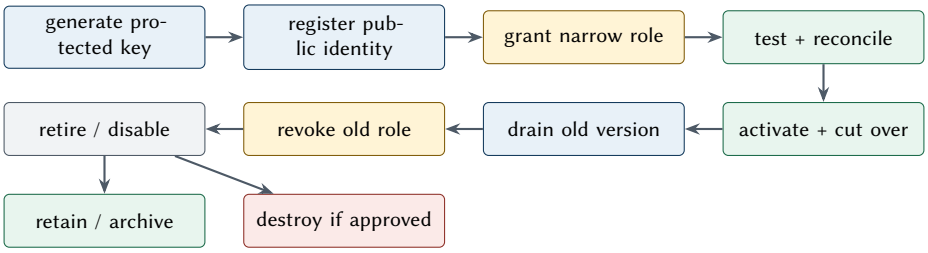

Figure 5. Evidence-gated key lifecycle and cross-boundary rotation.
Reading order: generation occurs inside the provider boundary; only public identity enters the registry. Grant, test, activation, cutover, drain, revocation, and retirement require cross-boundary evidence. Retired material is disabled and then retained/archived or destroyed according to approved recovery and retention policy.

Table 5. Native key and digest requirements.

Use	Key/signature	Message treatment	Compatibility gate
EVM account	secp256k1 ECDSA	32-byte Keccak-256 digest through no-rehash/DIGEST mode; native recovery metadata	Provider supports secp256k1 and exposes enough signature material for canonical low-s encoding.
Solana account	Ed25519	Exact serialized message through pure raw-message semantics; no prehash or extra SHA pass	Provider supports required message size, multi-signer assembly, public-key identity, and rejects digest-only or prehashed Ed25519 modes.
service identity	environment-selected asymmetric key	Protocol-defined challenge or mTLS transcript	Separate from transaction authority; rotate through workload-identity control.
evidence seal	approved signature/MAC	Canonical evidence envelope digest	Independent retention and verification path.

Recovery exercises prove policy, identity, provider access, on-chain grants, and reconciliation—not merely that a backup exists.

Backup treatment depends on technology: an HSM cluster may use provider-controlled secure backup; MPC may distribute independently protected shares; a custody service may offer contractual recovery. Recovery material receives equal or stronger control than an active key. Record quorum, geography, personnel separation, restore time, cryptographic verification, and destructive-test constraints.

Algorithm names are insufficient: managed KMS products differ on curve, RAW versus DIGEST input, message-size limits, DER versus native encoding, public-key export, latency, quotas, regional availability, audit events, and attestation. Confirm each path with provider documentation and whole-object vectors. (Amazon Web Services 2026c; Amazon Web Services 2026a; Amazon Web Services 2026b; Google Cloud 2026; Microsoft 2026; OASIS 2023; Josefsson and Liusvaara 2017)

Workload identity should be short-lived, audience-bound, and tied to the signer deployment, using platform identity or SPIFFE-style workload identity rather than static cloud credentials. Kubernetes service accounts require explicit audience and minimal RBAC. (SPIFFE Authors 2026; Kubernetes Authors 2026) A local signer uses disposable fixture keys and native libraries; a mock HSM models quotas, denials, malformed signatures, timeouts, and rotation. Both fail startup in a production profile.

Table 6. HSM, MPC, and custody decision surface.

Choice	Strength	Critical diligence	Typical boundary
self-managed HSM	Direct policy and audit control	operations skill, HA, backup, mechanisms, capacity, attestation	institution operates signer and hardware estate
cloud HSM/KMS	Managed durability and identity integration	algorithm/encoding fit, quotas, regional dependency, admin separation	institution operates signer; cloud protects key
MPC service	distributed authority and workflow options	share lifecycle, protocol/version, recovery, evidence, vendor control plane	co-managed or hosted signing workflow
qualified custody/platform	integrated policy, approvals, and operations	legal/control model, transaction semantics, exit/export limits, SLAs	provider controls substantial signing operation
local/mock	deterministic development and negative tests	production lockout, fixture isolation, semantic fidelity	non-production only

Implementation reference. Contextual signer contract — `digital-banking@ e921fcb1877b46a6881437f46b1a6ebfa115ae58`; module `application`; package `io.github.johnwhitton.digitalbanking.application.port`; `application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/SignerPort.java`; status: **interface only**.

Implementation notes. `SignerPort`, `SigningRequest`, and the actual pinned result type `SigningDecision` exist as an interface contract. No `SigningResult` type exists, and there is no dedicated signing service, contextual policy runtime, key registry, HSM/MPC/custody adapter, rotation workflow, or production signer.

Continue the thread. Volume I: *Security architecture—Key roles and signer policy.* Volume III planned chapter: signing, custody, and wallet boundary. ADRs: ADR-011, ADR-012, and ADR-014. Engineering modules: `shared/engineering/custody/signer-port-and-signing-boundary.md` and `shared/engineering/wallets/README.md`. Implementation: `application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/SignerPort.java`.

Part IV.
Part IV — EVM Engineering

4. EVM engineering

Why this chapter. EVM compatibility does not remove nonce, fee, signature, receipt, reorg, or contract-authority risk. The adapter must preserve those native semantics behind the shared application boundary.

4.1. Adapter and contract boundary

Use Web3j behind application ports for Java JSON-RPC, ABI encoding/decoding, transaction assembly, receipt handling, and signature utilities. Pin versions and compare its encoded output with independent vectors. Provider SDK objects must not enter the domain. Solidity contracts are a separate security product: build and test them with Foundry, publish verified source and compiler settings, and consume a reviewed ABI artifact in Java. (Web3 Labs 2026; Foundry Contributors 2026; Solidity Authors 2026a)

Exact pinned excerpt: shared `ChainPort` operations

`digital-banking@e921fcb1877b46a6881437f46b1a6ebfa115ae58 · application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/ChainPort.java`

```
public interface ChainPort {

    ChainCapabilities capabilities(String routeVersion);

    PreparedAttempt prepare(TokenOperation operation, OperationAttempt attempt);

    SubmissionResult submitOnce(SignedAttempt signedAttempt);

    InquiryResult inquire(AttemptIdentity attemptIdentity);

    Observation observe(ObservationRequest request);
```

These methods separate preparation, one submission call, ambiguity inquiry, and observation. The shared port does not erase the EVM and Solana payload, lifetime, signing, or finality rules that their adapters must enforce.

ERC-20 does not itself define who may mint or burn; the implementation must define authority, events, pause/deny behavior, supply invariants, and administrative change. Use explicit roles and least privilege, preferably separating routine mint/burn from pause, role administration, and upgrade authority. Require quorum/timelock or an equivalent governed path for high-impact changes, and monitor role events independently. OpenZeppelin components reduce reinvention but do not prove policy, integration, or upgrade safety. (Vogelsteller and Buterin 2015; OpenZeppelin 2026; Solidity Authors 2026b)

4.2. Construction, nonce, fees, and signing

For an EIP-1559 type-2 transaction, bind chain ID, nonce, max priority fee, max fee, gas limit, destination, value, calldata, and access list before authorization. Chain ID is a configured and runtime-verified replay domain; the signing service refuses a mismatch. Reserve nonces atomically per chain/account with an attempt ID and lease/evidence state. Compare local reservation, `eth_getTransactionCount` views, pending transactions, and receipts during recovery. Never allow two workers to allocate the same nonce. Once signing may have occurred,

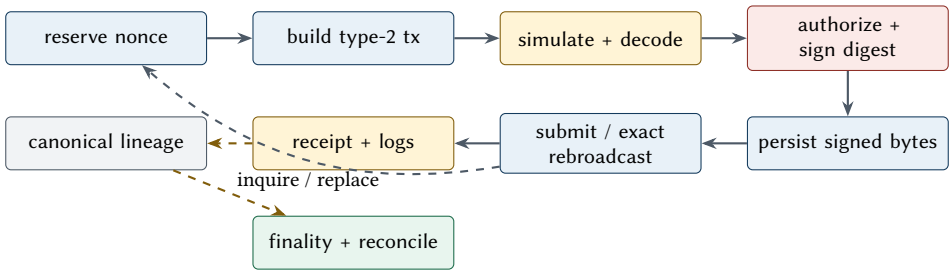

Figure 6. EVM transaction, replacement, observation, and reconciliation lifecycle.
Reading order: durable nonce reservation precedes construction, simulation, signing, persistence, submission or exact-byte rebroadcast, receipt/log observation, canonical-lineage evaluation, and reconciliation. Any fee replacement is a linked new attempt. Evidence status: planned; no EVM adapter exists at the pinned implementation commit.

ownership of the chain ID, sender, and nonce survives worker-lease expiry. Lease expiry transfers processing ownership only; reassignment requires proof that no signed or submitted effect exists, or an explicitly linked same-nonce replacement/cancellation attempt. (Ethereum Foundation 2026a; Buterin et al. 2019; Buterin 2016)

Simulation uses the exact sender, destination, value, calldata, and a conservative block context. Decode revert data and expected events, but remember that simulation is not execution and the state can change. Gas policy separates estimated gas, capped gas limit, base-fee conditions, tip, total maximum native fee, and business cost allocation. Require the maximum fee per gas to be at least the maximum priority fee, enforce the configured base-fee/inclusion policy, and bound gas limit times maximum fee per gas. Reject an unexpectedly payable call or native-value transfer. (Ethereum Foundation 2026b)

Fee replacement is a linked new attempt using the same account/nonce and semantically identical business call, with policy-bounded fee changes. Cancellation is another transaction and is not guaranteed to win. Persist both attempts and observe which transaction, if any, became canonical. Do not mutate or discard the earlier signed bytes.

The EVM account signature is ECDSA over secp256k1. Hash the typed transaction according to its envelope with Keccak-256; do not substitute SHA-3. A provider may return DER-encoded integers or fixed-width components. Strictly parse minimal positive DER integers, left-pad r and s to 32 bytes, validate $1 \leq r, s < n$, and require $s \leq n/2$ for an Ethereum transaction. If an adapter converts s to $n - s$, it flips `yParity`, then recovers and requires the registered address. Never guess parity from DER or from the sign of an integer. Verify the signature over the reconstructed digest, then serialize the complete signed envelope. (Buterin 2015; Standards for Efficient Cryptography Group 2010; Pornin 2013)

Persist the unsigned semantic fields, canonical signing digest, signed transaction bytes, transaction hash, key version, signature components, policy decision, and replacement lineage before broadcast. Persisting signed bytes makes an identical rebroadcast safe; private keys must never enter application persistence. Signed bytes are replay-capable authorization artifacts: encrypt and integrity-protect them under controlled retention, and never place raw unsigned or signed payloads in logs.

4.3. Submission, observation, finality, and reconciliation

Broadcast immutable signed bytes through a submission adapter. A returned hash or RPC success is an acknowledgement, not finality. Inquire by the locally derived transaction hash after timeouts. Observe receipt status, block hash/number, transaction index, gas usage, effective price, and contract logs. Decode the expected event and compare asset, amount, source/destination, operation reference where represented, and contract address. A successful receipt without the expected business event is an exception, not success.

Track canonical lineage and re-check receipts across the profile's confirmation and finality policy. A receipt may disappear or move in a reorganization. For L2s, distinguish sequencer acceptance, L2 inclusion, data availability, proof/fault window, and L1 settlement evidence; do not reuse an L1 confirmation count blindly. Reconciliation compares the authoritative internal ledger and operation intent with the winning canonical transaction and events, including fee and any replacement siblings.

4.4. Testing and security gate

Foundry unit, property, fuzz, invariant, and fork tests cover access control, supply conservation, pause/upgrade behavior, malformed calls, event accuracy, and reentrancy/external-call hazards. Anvil integration tests cover Web3j encoding, type-2 signing vectors, nonce contention, replacement, dropped transactions, reverts, event decoding, and synthetic reorg/recovery. Whole-object vectors compare unsigned fields, digest, r, s, parity, signed bytes, hash, receipt, and event. Independent smart-contract and signing reviews are release gates.

Implementation reference. EVM chain boundary — `digital-banking@ e921fcb1877b46a6881437f46b1a6ebfa115ae58`; module `application`; package `io.github.johnwhitton.digitalbanking.application.port`; `application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/ChainPort.java`; status: **interface only**.

Implementation notes. `ChainPort` is interface only. No Web3j adapter, Solidity contract, Foundry suite, nonce manager, EVM signer encoding, submission, observation, or reconciliation runtime exists; EVM execution is planned. Nonce ownership survives worker-lease expiry because a possible signature or submission outlives process ownership.

Continue the thread. Volume I: *Component contracts and ownership*, *Payment lifecycle and flow catalog*, and *Failure model and containment*. Volume III planned chapter: EVM adapter and example contract. ADRs: ADR-005, ADR-007, and ADR-008. Engineering modules: `shared/engineering/smart-contracts/evm-contract-engineering.md` and `shared/engineering/custody/evm-signing.md`. Implementation: `application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/ChainPort.java`.

Part V.
Part V — Solana Engineering

5. Solana engineering

Why this chapter. Solana's account model, ordered message, blockhash lifetime, compute policy, and fork commitments are not EVM concepts with different names. A safe adapter preserves the native model while sharing only business-level ports.

5.1. Preserve native semantics

The Java control plane owns durable business state and calls a Solana-specific adapter. Native program logic, when needed, is Rust; Anchor can supply account constraints, discriminators, instruction scaffolding, and testing conventions, but those generated checks must be reviewed. Do not create a custom program merely to imitate an EVM contract. Prefer audited SPL Token or Token-2022 facilities when their authority and extension models meet the requirement, and add a program only for a proven missing invariant. (Solana Foundation 2026d; Anchor Contributors 2026; Solana Program Library Contributors 2026; Solana Foundation 2026e)

Accounts are explicit inputs. The adapter resolves program IDs, mint, token accounts, associated token accounts, fee payer, authorities, sysvars, and any program-derived addresses (PDAs); it records why each account is required and whether it is signer/writable. A PDA is derived from seeds plus a program ID and cannot sign as an ordinary keypair. Through `invoke_signed`, the calling program can add signer authority only for a PDA derived with that calling program ID; this is distinct from an account's data-owner field. Policy completely decodes the top-level message, ordered account metas, and instruction data, and permits only reviewed program IDs and CPI surfaces. Dynamic CPIs are created during execution and cannot be proven from static decoding: simulation inner instructions are evidence, not proof of every runtime path. Program constraints, runtime privilege checks, independent observation, and negative tests must reject account substitution, unexpected CPI targets, or privilege escalation. (Solana Foundation 2026a; Solana Foundation 2026d)

Before signing a token transfer, mint, or burn, reconstruct an associated token account from the wallet authority, selected Token or Token-2022 program ID, and mint. Validate the account's program owner and decoded mint/token authority against those exact inputs; reject an absent, substituted, wrong-program, wrong-mint, or wrong-authority account. A non-associated token account is allowed only through a deliberate profile rule with equivalent validation and evidence.

5.2. Message construction, lifetime, and signing

A transaction contains signatures and a compiled message. The message binds header, ordered account keys, recent blockhash, and compiled instructions. Ed25519 signs the exact serialized message; do not apply EVM hashing, DER conversion, recovery parity, or low-s rules. The signing service reconstructs the same bytes independently and verifies every returned signature against the expected public key. Signatures are assembled in the header-required signer order. (Solana Foundation 2026g; Josefsson and Liusvaara 2017)

Fetch a recent blockhash with its last valid block height and persist both on the attempt. Height, not wall-clock estimation alone, determines validity. Before first send and any rebroadcast, check current validity. If the blockhash expires, a refreshed blockhash produces a different message and therefore requires a new linked attempt and new signatures. Never attach an old signature to a new message. (Solana Foundation 2026c)

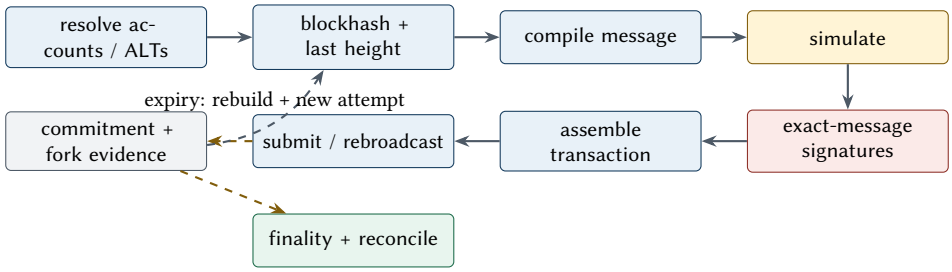

Figure 7. Solana message lifetime, signing, commitment, and reconciliation lifecycle.
Reading order: resolve accounts and lookup tables, bind blockhash lifetime, compile and simulate one message, collect signatures in header order, assemble and submit, then observe commitment and forks independently. Expiry changes the message and requires a new attempt. Evidence status: planned; no Solana adapter exists at the pinned commit.

The fee payer is the first signer and may be distinct from mint/burn authority. Multiple signers must all authorize one immutable message; missing or duplicate signature slots fail closed. Versioned transactions and address lookup tables (ALTs) reduce static account-key pressure but add lookup-table identity/state to policy and evidence. Resolve and pin the table contents used to compile, and reject deactivated, extended-after-policy, or unexpected tables as the profile requires. (Solana Foundation 2026h; Solana Foundation 2026b)

Estimate compute through simulation, then set bounded compute-unit limit and price instructions under fee policy. Simulation must use the exact message shape and replaceable lifetime fields only through an explicit pre-sign construction stage. Record logs, units consumed, return data, and account changes needed for policy. Simulation success is not a durable reservation of state. (Solana Foundation 2026f)

5.3. Submission, commitment, indexing, and reconciliation

Persist message bytes, all signatures, full transaction bytes, signature identity, blockhash lifetime, key versions, lookup tables, simulation/policy evidence, and attempt lineage before submission. Rebroadcast identical bytes while valid; an ambiguous response is resolved by signature inquiry and independent observation. Once expired with no evidence, transition under recovery policy before rebuilding.

Solana commitment levels describe node observations of fork/confirmation state; they are not the four business finalities. Record slot, blockhash, parent lineage, commitment, transaction error, logs, balances, token balance changes, and decoded instructions. A Geyser/indexing feed can improve latency and history, but it is an observation adapter whose completeness, ordering, replay, and failure domain require evidence. Cross-check critical state with RPC or an operated verifier. (Solana Foundation 2026g; Anza 2026)

Reconciliation verifies the intended mint/burn, mint address, token program, authority, source and destination token accounts, exact amount/decimals, fees, and canonical transaction result against the authoritative internal ledger. Token-2022 extensions such as transfer fees, hooks, confidential features, default account state, or permanent delegation change semantics and require a profile decision; unsupported extensions fail closed.

5.4. Testing and security gate

Rust/Anchor unit and property tests cover account ownership, signer/writable constraints, PDA seeds, CPI targets, arithmetic, duplicate accounts, and authority changes. A `solana-test-validator` suite covers Java message construction, associated-token-account creation, multi-signer ordering, blockhash expiry, versioned messages, ALTs, compute limits, priority fees, failure logs, rebroadcast, and observation/reconciliation. Golden vectors include account order, message bytes, each signature, transaction bytes, and decoded outcome. Security review focuses on account substitution, confused-deputy CPIs, unchecked owners, signer privilege, PDA derivation,

Table 7. Native engineering differences that adapters must preserve.

Concern	EVM	Solana
execution input	destination, value, calldata, gas, chain ID, nonce	programs, ordered account metas, instructions, blockhash, header
replay/lifetime	account nonce and chain ID	recent blockhash plus last valid block height
signature	secp256k1 ECDSA over Keccak-derived digest; r, s, parity, low-s	Ed25519 over exact serialized message; ordered signature array
concurrency	atomic nonce ownership per account	writable-account locks and blockhash lifetime
fee policy	gas limit, base fee, priority fee, max fee	fee payer, compute units, compute-unit price, signature/base fees
state evidence	receipt, status, logs, canonical block lineage	transaction metadata, instructions/logs, account/token deltas, slot/fork commitment
replacement	same nonce, linked fee replacement	expired/new blockhash means new message and linked attempt
program model	contract storage and calls	explicit accounts, PDAs, instructions, CPIs

extension behavior, and upgrade authority. Negative cases include wrong ATA wallet, token program, mint, decoded authority, account owner, non-ATA use without profile approval, and unexpected CPI program or privilege.

Implementation reference. Solana chain boundary — `digital-banking@ e921fcb1877b46a6881437f46b1a6ebfa115ae58`; module `application`; package `io.github.johnwhitton.digitalbanking.application.port`; `application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/ChainPort.java`; status: **interface only**.

Implementation notes. The generic `ChainPort` is interface only. No Solana Java adapter, Rust/Anchor program, token integration, signer path, local-validator suite, Geyser adapter, or reconciliation runtime exists. Solana execution is planned; a custom Rust program is intentionally excluded until a native token/program gap is demonstrated. Blockhash expiry changes the message, so recovery rebuilds and re-authorizes a linked attempt rather than replaying stale bytes.

Continue the thread. Volume I: *Technology evaluation* and *Failure model and containment.* Volume III planned chapter: Solana adapter and example program. ADRs: ADR-005, ADR-007, and ADR-008. Engineering modules: `shared/engineering/smart-contracts/solana-anchor-engineering.md` and `shared/engineering/custody/solana-signing.md`. Implementation: `application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/ChainPort.java`.

Part VI.
Part VI — Submission and Observation

6. Submission and observation

Why this chapter. Submission asks a provider to relay bytes; observation establishes what the network made canonical. Keeping those paths operationally independent is what turns ambiguous responses into recoverable evidence rather than guesses.

6.1. Separate the phases and failure domains

Prepare, sign, submit, and observe are four durable phases. Preparation creates native semantic fields, lifetime constraints, simulation, and policy evidence. Signing authorizes one immutable payload. Submission sends persisted bytes. Observation independently asks what the network made canonical. Each phase has an input/output digest, attempt identity, owner, timeout, and evidence.

Invariant. A submission acknowledgement is not settlement. An RPC response, transaction hash, Solana signature, broker acknowledgement, or HTTP success cannot by itself advance blockchain, legal settlement, customer-visible, or accounting finality.

A managed RPC is a replaceable adapter. Record provider, endpoint class (not secret URL), request identity, native response, latency, and error taxonomy. Use an independent secondary provider for critical observation and optionally an operated read/verifier node when its cost and operational burden are justified. Independence includes organization, region, account/control plane, network path, and underlying infrastructure; two brands may share a hidden failure domain.

6.2. Ambiguity, duplicates, polling, and streams

Derive the EVM transaction hash or Solana signature locally before submission. If the request times out, inquire by that identity; rebroadcast only identical signed bytes within native lifetime. Duplicate broadcast may be harmless at the protocol layer but must still be recorded. A changed EVM fee envelope or Solana blockhash is a new attempt. Provider errors are classified as definitive rejection, retryable pre-effect, ambiguous after possible effect, rate/capacity, authentication/configuration, or unsupported method.

Polling is the durable correctness baseline: use adaptive backoff, jitter, request budgets, canonical-lineage checks, and a maximum state age that raises recovery work. Streaming reduces latency but requires cursor persistence,

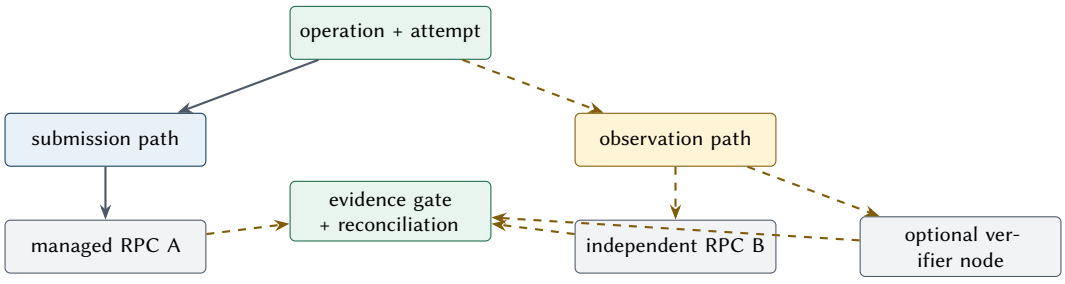

Figure 8. Failure-independent submission and observation paths.
Reading order: the attempt is submitted through one provider, while a logically and operationally separate observer gathers native evidence through another provider and optionally an operated verifier. Reconciliation compares sources before business finality advances.

Table 8. RPC and observation sourcing choices.

Source	Use	Evidence needed	Exit concern
primary managed RPC	submission and routine reads	methods, limits, consistency, regions, incident history, auditability	portable client, endpoint/config switch, queued-work recovery
secondary managed RPC	independent inquiry/observation	failure-domain analysis, cross-provider equivalence, historical depth	tested failover and data comparison
self-hosted read/verifier	stronger derivation and replay control	sync integrity, upgrades, capacity, snapshots, peer/network health	operational staffing and restore time
indexer/stream	low-latency events and history	ordering, completeness, replay cursor, fork behavior, schema/version	raw-native fallback and rebuild procedure
archive source	historical reconciliation/investigation	retention, provenance, query reproducibility, cost	export format and retention copy

reconnect/backfill, duplicate handling, fork reversal, schema compatibility, and comparison with polling. No in-memory subscription is the sole record.

Evidence is append-only and source-qualified. Store native identifiers and structured facts, plus a digest or controlled raw response when needed for investigation. An evidence gate advances blockchain finality only after the configured independent sources, canonical lineage, and chain- specific policy agree. Reconciliation then compares final native outcome with ledger postings, business amount, authority, fees, and sibling attempts. Disagreement creates a break; it does not select the more convenient source.

Provider exit is exercised, not asserted. Maintain a provider-neutral semantic port, native golden vectors, alternate endpoint configuration, exportable historical evidence, rate-limit behavior, and a runbook for credentials, DNS/network rules, queue drain, ambiguity recovery, and dual observation. A quarterly or release-level drill should prove read failover without signing or moving value.

Implementation reference. Observation contract — `digital-banking@ e921fcb1877b46a6881437f46b1a6ebfa115ae58`; module `application`; package `io.github.johnwhitton.digitalbanking.application.port`; `application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/ChainPort.java`; status: **interface only**.

Implementation notes. `ChainPort` includes observation request/result types, but the boundary is interface only. No submission adapter, managed-provider wiring, independent observer, poller/stream, verifier node, evidence gate, or reconciliation worker is implemented. Independent observation and reconciliation remain separate responsibilities: one qualifies native evidence; the other compares it with ledger and business truth.

Continue the thread. Volume I: *Failure model and containment* and *Four independent finalities*. Volume III planned chapter: infrastructure, deployment, and observability. ADR: ADR-013. Engineering modules: `shared/engineering/infrastructure/blockchain-infrastructure.md` and `shared/engineering/observability/README.md`. Implementation: `application/src/main/java/io/github/johnwhitton/digitalbanking/application/port/ChainPort.java`.

Part VII.
Part VII — Infrastructure and Operations

7. Infrastructure and operations

Why this chapter. Durable state only helps when workers, queues, credentials, providers, and operators can recover it after process and regional failure. Operations therefore owns progress, ambiguity age, and evidence retention rather than merely service uptime.

7.1. Reference topology and durable dependencies

Deploy the admission API, recovery/worker fleet, signing service, and observer as independently scalable and permissioned workloads. PostgreSQL holds authoritative operation coordination, ledger-facing references, outbox/inbox, and evidence indexes. Messaging carries notifications and work hints; losing a message must not lose the operation. Protected cryptography, submission RPC, independent observation, and durable audit export occupy explicit trust and failure boundaries.

PostgreSQL operations cover high availability, point-in-time recovery, encrypted backups, connection limits, query/lock monitoring, bloat/vacuum, migration rehearsal, and periodic restore tests. Define RPO/RTO separately for the ledger, operation state, evidence, and disposable telemetry. A restored database is not ready until outbox leases, in-flight attempts, signer requests, provider submissions, and reconciliation cursors have been recovered safely.

Messaging configuration sets partition/ordering scope, retention, producer/consumer identity, encryption, quotas, and replay procedure. Backpressure begins at admission and worker claims: publish queue depth, oldest-state age, database saturation, signer capacity, provider limits, and reconciliation backlog. Apply participant and operation-class quotas. Shed noncritical inquiry and reporting work before safety-critical recovery.

Secrets are references resolved through workload identity, not embedded values in configuration, images, or logs. Network policy permits the signer to reach only required identity, policy, HSM, and audit endpoints. HSM connectivity has bounded pools/timeouts, mechanism negotiation, quota telemetry, and explicit fail-closed behavior. Provider keys, RPC credentials, database roles, and audit-export credentials rotate independently.

7.2. Telemetry, audit, SLOs, and alerts

Traces show causal work across asynchronous boundaries through operation/attempt identifiers and span links. Metrics include admission success/conflict, transition latency and age, outbox claim and delivery, signer

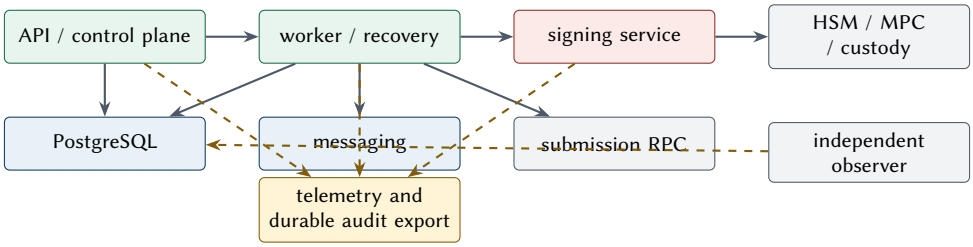

Figure 9. Reference deployment topology and isolated authority boundaries.
Reading order: the API durably accepts work; workers recover and coordinate; a separately deployed signer reaches protected cryptography; submission and observation use independent paths; durable audit evidence and disposable telemetry remain distinct. Exact products, regions, and scaling are profile decisions.

decision/latency/quota, submission ambiguity, confirmation/finality lag, fork/reorg events, provider disagreement, and reconciliation breaks. Structured logs describe diagnostics; never place secrets, key material, unredacted signed payloads, or full participant data in them. Durable audit records policy facts, approvals, key version, payload digest, provider identity, result, and evidence links under stricter access/retention.

Service-level objectives are user and safety outcomes, not only uptime: durable acceptance availability/latency, maximum age to first authorized attempt, maximum ambiguous-state age, observation freshness, reconciliation completion, and maximum break age. Error budgets exclude neither provider incidents nor slow finality; dependencies inform the budget and capacity plan. Alerts are actionable and paired with a runbook: database failover/PITR risk, queue/state-age breach, signer denial anomaly, provider divergence, expired native lifetime, finality regression, stuck migration, capacity exhaustion, and ledger/native mismatch.

7.3. Incident, disaster, and provider failure

Incident command separates containment, business decision, technical recovery, evidence capture, and communication. Preserve signed bytes and observations before remediation. Security response can deny signing, revoke workload identity, pause a contract/program, rotate on-chain authority, or stop admission; each action needs named approval and recovery evidence. Never delete or rewrite in-flight history to make dashboards green.

Regional failure procedure promotes services and database only after fencing the old writer, validating restore/replication point, re-establishing workload and signer identity, and recovering leases/cursors. Provider failure switches new submissions only after classifying ambiguity for old ones; observation moves independently. Disaster exercises include lost region, corrupt backup, broker replay, unavailable signer provider, degraded RPC, observer disagreement, and credential revocation.

7.4. Deployment, migration, rollback, and runbooks

Release artifacts are immutable and identified by source, dependency/SBOM, configuration schema, database migration, ABI/program ID, policy bundle, and test evidence. Progress through development, native local chain, integration, non-value canary, bounded pilot, and production profiles. A canary must not share an unsafe nonce allocator or alternate ledger truth. Migrations use compatibility windows; application rollback is rehearsed; signer/policy and on-chain changes have independent rollback or forward-fix plans.

The runbook catalog covers: operation inquiry; ambiguous submission; EVM nonce conflict and fee replacement; Solana blockhash expiry; expected-event absence; reorg/fork regression; signer denial or malformed signature; key compromise/rotation/revocation; HSM/provider outage; RPC failover; indexer replay; outbox/inbox recovery; reconciliation break; database failover/PITR; broker replay; capacity/backpressure; schema migration/rollback; contract/program pause/upgrade; evidence export; and security incident. Each runbook states trigger, prerequisites, authority, safe commands, validation, rollback, escalation, and evidence to retain.

Implementation reference. PostgreSQL persistence and pending outbox — `digital-banking@ e921fcb1877b46a6881437f46b1a6ebfa115ae58`; module `adapters/persistence-postgres`; package `io.github.johnwhitton.digitalbanking.persistence.postgres`; `adapters/persistence-postgres/src/main/java/io/github/johnwhitton/digitalbanking/persistence/postgres`; status: **partially implemented**.

Implementation notes. PostgreSQL request persistence and pending outbox records exist, but no publisher/consumer, workload identity, HSM/RPC connectivity, OpenTelemetry wiring, dashboards/alerts, disaster-recovery automation, or operational runbooks exist. A pending outbox row is durable intent, not delivery; workers must lease, publish, record outcome, replay safely, and expose aged work. Deployment remains planned.

Continue the thread. Volume I: *Consistency: narrow atomicity, end-to-end recoverability* and *Messaging and event contracts*. Volume III planned chapter: local environment and infrastructure examples. ADRs: ADR-004, ADR-013, and ADR-014. Engineering modules: `shared/engineering/deployment/README.md` and `shared/engineering/infrastructure/README.md`. Implementation: `adapters/persistence-postgres/src/main`.

Part VIII.
Part VIII — Testing and Verification

8. Testing and verification

Why this chapter. A passing test is useful only when its source revision, environment, oracle, and limitation are visible. Layered verification connects architecture claims to the cheapest evidence that can actually falsify them.

8.1. Evidence layers

Testing proves bounded claims. Each requirement maps to one or more tests, source revision, environment, fixture digest, result, and retained artifact. Passing unit tests cannot establish provider compatibility or production resilience; a pilot cannot compensate for missing invariant tests. Use the cheapest deterministic layer that can falsify the claim, then add boundary and failure evidence.

Unit tests cover exact amounts, canonical commands, identity equality, state transition guards, retry authorization, and corrections. Whole-object tests use canonical serialized commands, outbox envelopes, EVM envelopes/signatures/hashes, Solana message/signature/transaction bytes, and evidence envelopes. Property/fuzz tests generate boundary amounts, malformed encodings, transition sequences, DER integers, account orderings, and duplicate/reordered messages while preserving a reproducible seed.

Contract tests run against every port implementation: repositories, outbox/inbox, policy, SignerPort, EVM/Solana adapters, submission, and observation. Testcontainers supplies real PostgreSQL and supported broker versions for migration, locking, isolation, lease recovery, uniqueness, replay, and upgrade tests. Do not replace these with an in-memory database whose transaction semantics differ. (Testcontainers 2026)

Anvil exercises Solidity/Foundry artifacts and the Java EVM path. A `solana-test-validator` exercises token/program and Java Solana paths. Deterministic fixtures declare chain genesis/config, contract/program artifacts, key fixtures, policy bundle, clock, RPC faults, and expected byte/evidence digests. Simulation/replay tests retain raw native inputs and decoded semantic comparisons. A local validator does not establish production cluster feature-gate, fee-market, managed-provider, or performance compatibility; those require selected- environment evidence.

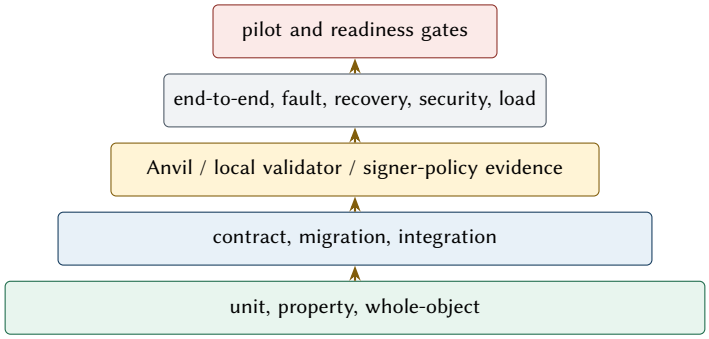

Figure 10. Testing layers grow in integration cost and evidence scope.
Reading order: cheap deterministic invariant tests form the base; contracts and real dependencies add boundary evidence; native local chains and signer policy prove encoding and lifecycle; system, fault, recovery, security, and load tests support bounded pilot and readiness decisions. No layer alone proves production readiness.

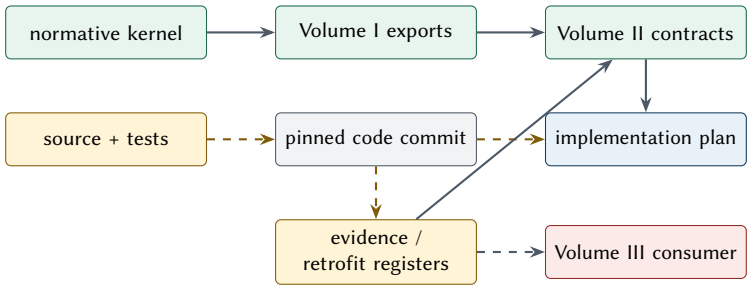

Figure 11. One-way architecture authority and bounded implementation feedback.
Reading order: authority flows from the kernel through Volume I exports and Volume II engineering contracts into plans and code. Source and tests at an exact commit supply bounded evidence upward through registers; they never redefine architecture. Volume III remains a future consumer.

8.2. Required fault and concurrency suites

Concurrency and idempotency tests race identical and conflicting admission keys, optimistic state updates, worker leases, outbox claims, inbox duplicates, signer requests, EVM nonce reservations, and Solana blockhash refresh. Prove one durable business effect and a complete audit trail under retries and process termination.

Signing-policy tests include valid native payloads plus wrong chain/program/contract, asset, amount, authority, destination, fee, nonce/blockhash, instruction/account, hidden native-value transfer, extra call/CPI, altered digest, stale policy, unauthorized key version, malformed provider signature, provider timeout, and replay. Key-rotation tests span grant, dual-read/drain, cutover, in-flight old version, revocation, provider loss, recovery, and on-chain verification.

Native failure tests include EVM nonce collision, underpriced replacement, competing transaction, revert, missing event, receipt disappearance, and reorg; and Solana expired blockhash, missing signer, account lock, compute exhaustion, priority-fee pressure, ALT change, program error, and fork commitment regression. Submission tests inject timeout before send, timeout after possible send, duplicate response, malformed response, provider disagreement, rate limit, stream gap, cursor loss, and stale observation. Reconciliation tests cover fee differences, partial/duplicate events, sibling attempts, correction/compensation, and material break escalation.

Fault injection kills processes between every durable step, pauses dependencies, exhausts pools, corrupts non-authoritative caches, revokes credentials, and delays/reorders messages. Load tests measure throughput, tail latency, state age, database locks, queue growth, signer quotas, provider rate limits, observation lag, and recovery after overload. A benchmark result is useful only with hardware/environment, dataset, native chain configuration, duration, warmup, error rate, and artifact version.

Security review covers threat model and authority separation; API/identity and participant scope; dependency/SBOM and secret scanning; signer payload decoding and algorithm encoding; smart contract or Solana program; provider administration; workload identity/network policy; audit integrity; incident and key recovery; and abuse/load cases. Findings are severity-ranked, owned, and linked to retest evidence.

8.3. Publication-to-code evidence

The implementation status ledger cites exact source/test paths at the pinned commit and records limitations. Publication acceptance tests check chapter/figure/table inventory, status vocabulary, signing invariants, exact EVM/Solana treatment, evidence fields, release identity, PDF geometry, fonts, links, references, source archive, and clean deterministic rebuild. A human visual review still checks clipping, density, reading order, captions, tables, bookmarks, and rendered symbols.

Implementation reference. Pinned implementation test evidence — `digital-banking@ e921fcb1877b46a6881437f46b1a6ebfa115ae58`; modules `domain`, `application`, `control-plane`, and `adapters/persistence-postgres`; `docs/IMPLEMENTATION.md`; status: **implemented**.

Implementation notes. The pinned implementation plan records 302 passing tests for its then-current suite. This review treats the count as a committed record, not a fresh execution. Source and committed tests support bounded domain, acceptance, HTTP/security-fixture, and persistence claims; they do not prove worker, chain, signer, provider, resilience, or production behavior.

Continue the thread. Volume I: *Evidence taxonomy and summary* and *Evidence-gated roadmap*. Volume III planned chapter: test and failure strategy. ADRs: ADR-004, ADR-005, and ADR-013. Engineering module: `shared/engineering/testing/README.md`. Implementation: `domain/src/test`, `application/src/test`, `control-plane/src/test`, and `adapters/persistence-postgres/src/test`.

Part IX.
Part IX — Performance, Batching, and Economics

9. Performance, batching, and economics

Why this chapter. Throughput is valuable only when each logical payment retains authority, attempt, cost, and reconciliation lineage. This chapter turns performance and batching choices into measurable safety and ownership gates without repeating Volume I's business analysis.

9.1. Measure end-to-end state age

Throughput is bounded by the slowest safety-critical resource: database serialization, queue and worker capacity, policy/approval, protected signing, native account/nonce contention, provider quota, chain inclusion, observation, or reconciliation. Measure per-stage service and wait time, tail latency, saturation, error/ambiguity rate, and recovery backlog. Admission throughput without downstream state-age limits merely converts an outage into durable debt.

Batching trades unit cost for latency, concentration, and recovery complexity. A batching policy sets maximum window, item/byte/account/gas/compute ceilings, per-participant fairness, value and risk limits, deterministic order, cancellation cutoff, and partial-failure semantics. Each logical payment retains identity and allocation evidence. No batch should make one malformed or unauthorized item invisible inside a bulk signature.

EVM ceilings include block and per-transaction gas, calldata, contract loop/storage behavior, nonce serialization, fee volatility, and L2 data/posting constraints. Solana ceilings include transaction/message size, account-key and ALT constraints, compute units, writable-account contention, signatures, blockhash lifetime, priority fees, and program behavior. Test actual encoded transactions; a spreadsheet estimate is not a protocol gate.

Retry and partial failure are native-specific. An EVM batch call may revert atomically or implement per-item outcomes; either design needs explicit event and ledger semantics. A Solana transaction is atomic across its instructions, while a sequence of transactions is not. A retry never creates a second logical payment; it creates an attempt with native lineage and allocated cost.

9.2. Net settlement, payer models, and cost

Net settlement reduces external actions only when legal, liquidity, participant, timing, and exposure policies permit. Preserve gross obligations, netting set/window, cut-off, optimization version, residual positions, funding, external attempts, and allocation. A failed net action does not erase gross ledger obligations. Concentration and delayed settlement must be priced as risk, not hidden as throughput improvement.

Payer models include institution fee payer, participant fee payer, sponsored/paymaster path, or a hybrid. Record who is economically charged separately from the native account that pays. Policy covers funding, limits, fee-token volatility, stuck-account recovery, abuse, allocation, and participant disclosure.

Marginal cost is the incremental native fee and per-request provider usage for one additional logical payment. Allocated cost also includes signer/custody, RPC/indexing, operated nodes, database/messaging, observability, engineering/on-call, security/compliance, capital/liquidity, and shared batch overhead. Report cost per logical payment with percentiles and separate fixed, variable, and risk allocations. Do not divide a successful batch fee by submitted items when some failed or were retried.

The shared chain cost analysis and economics module are bounded inputs, not universal forecasts. Their scenario assumptions, prices, workloads, and access dates travel with any reused number. This companion imports engineering consequences only and does not repeat Volume I's business-model analysis.

9.3. Benchmark gates

A stage advances only when a repeatable benchmark meets explicit gates for: admission and transition tail latency; maximum state and ambiguity age; sustainable logical payments per second; database/queue/signer/provider headroom; native success and replacement/expiry rate; observation and reconciliation lag; recovery time after a controlled fault; cost per successful logical payment; and zero violated money, authority, idempotency, or evidence invariants. Mark missing dimensions *not evaluated*, not zero.

Implementation reference. Performance and economics evidence boundary — `digital-banking@ e921fcb1877b46a6881437f46b1a6ebfa115ae58`; implementation plan and module inventory; `docs/IMPLEMENTATION.md`; status: **unresolved**.

Implementation notes. Only durable request acceptance and database persistence can support meaningful local control-plane measurements at the pinned commit. There is no native execution, signer, observation, reconciliation, production topology, or representative economics benchmark. Batch ownership must retain item identity, deterministic membership, attempt lineage, fee/cost allocation, partial-failure policy, and an accountable reconciliation owner. Results beyond the control plane are planned or unresolved.

Continue the thread. Volume I: *Transaction cost and operating economics—Batching, aggregation, and net settlement.* Volume III planned chapter: load, soak, and failure testing. ADRs: ADR-006, ADR-007, and ADR-015. Engineering modules: `shared/engineering/chains/transaction-cost-analysis.md` and `shared/engineering/economics/README.md`. Implementation: `docs/IMPLEMENTATION.md`.

Part X.
Part X — Implementation Roadmap

10. Implementation roadmap

Why this chapter. The shortest safe route is not simultaneous multi-chain and provider breadth; it is a sequence in which each new authority is introduced only after the durable state and evidence needed to recover it exist.

The roadmap is evidence-gated, not date-gated. Each stage preserves operation/attempt/evidence lineage and may be stopped without claiming later capabilities.

Table 9. Stages and minimum exit evidence.

Stage	Deliverable boundary	Minimum exit evidence
prototype	exact domain model and state transitions	invariant/property tests; canonical identities and exact money; transition audit
durable acceptance	API, idempotency, repository, migrations	committed operation and replay/conflict evidence; empty/prior database migration
outbox	transactionally coupled publication intent and recovery	claim/lease/delivery/replay tests; no lost or duplicate business effect
workers	recoverable lifecycle progression	process-kill tests at each boundary; bounded state age; owned ambiguity
local signer	contextual policy with fixture keys	independent decode/digest reconstruction; negative policy and native vector suite
mock HSM	provider-shaped denial, timeout, quota, and rotation behavior	identity/admin separation; malformed-result, unknown-outcome, recovery, and audit tests
Ethereum	Anvil, Web3j, reviewed fixture contract, nonce manager	exact type-2 vectors; nonce/replacement/reorg; contract-event and independent-observation evidence
Solana	local validator, native adapter, token/program fixtures	exact message/signature vectors; blockhash expiry, account locks, forks, and independent-observation evidence
observation	independent source and optional verifier/indexer	canonical/fork lineage; replay/backfill; provider disagreement; freshness SLO
reconciliation	ledger/native comparison and break workflow	happy/mismatch/sibling/correction cases; ownership, aging, and finance-operations acceptance
production signer	selected HSM/MPC/custody adapter and protected operations	compatibility, quotas/latency, provider outage/ambiguity, rotation/recovery, audit, and exercised exit
operational hardening	production-shaped deployment, telemetry, recovery, and runbooks	failure injection; alert and SLO coverage; backup/restore, failover, rollback, and runbook exercises
security	reviewed identities, policies, software supply chain, and incident controls	threat-model closure; penetration and dependency evidence; least privilege; key and incident exercises

Stage	Deliverable boundary	Minimum exit evidence
performance	measured end-to-end capacity and cost under failure	sustained/peak load, latency, queue age, provider quotas, finality lag, and unit-cost gates
pilot	bounded participants, assets, value, and support exposure	approved limits; monitored transactions; daily reconciliation; incident and customer-remedy readiness
production readiness	governed release and operating acceptance	security, finance operations, risk, compliance, SRE, support, DR, rollback, and residual-risk sign-off

Durable acceptance has passed at the pinned commit: domain, request acceptance, and persistence evidence exist. The outbox stage is in progress because a pending outbox record is committed, but recoverable publication, claim/lease/delivery, and replay gates have not passed. The lifecycle worker and all native, signing, observation, reconciliation, and production-operation stages remain incomplete. Re-run every claimed gate from the exact commit and environment before reliance.

The next engineering increment should complete recoverable outbox publication and lifecycle driving without adding chain or signer authority. Then implement one native local-chain vertical slice with contextual signing and independent observation, preserving ports that can support the other chain. Avoid simultaneous provider procurement, multi-chain breadth, custom chain, and production deployment before the state/evidence spine is proven.

Network selection evaluates Ethereum, Base, Solana, and independently Arbitrum and Optimism under asset support, finality semantics, fee/throughput, provider diversity, operational burden, compliance/evidence, and exit. A future organization-specific L2 is a separate build-versus-buy and security/governance decision. OP Stack, Arbitrum Orbit, Caldera, and Conduit are candidates for that later evaluation, not approved architecture. (Optimism Collective 2026; Offchain Labs 2026; Caldera 2026; Conduit 2026)

Java remains the control-plane baseline. TypeScript/Viem, Rust/Alloy, Go/Geth APIs, and Python web3.py can be useful for tooling, native services, independent vector generation, or operations; introduce another language only where it materially improves a bounded capability and its build, security, on-call, and evidence cost is accepted. (Viem Contributors 2026; Alloy Contributors 2026; go-ethereum Authors 2026; web3.py Contributors 2026)

Implementation reference. Current roadmap position — `digital-banking@ e921fcb1877b46a6881437f46b1a6ebfa115ae58`; domain, application, control-plane, and PostgreSQL adapter modules; `docs/IMPLEMENTATION.md`; status: **partially implemented**.

Implementation notes. The pinned implementation has passed durable acceptance and has a pending outbox insert, but not recoverable publication or workers. The outbox stage is therefore in progress, not complete. It cannot claim local signer, mock HSM, Ethereum, Solana, observation, reconciliation, production signer, hardening, security, performance, pilot, or production-readiness capability. Each later gate must cite a fresh exact commit and environment.

Continue the thread. Volume I: *Evidence-gated roadmap.* Volume III planned chapters: module architecture, EVM, Solana, signing, runtime, tests, and production-gap register. ADRs: ADR-006, ADR-014, and ADR-015. Engineering modules: `shared/engineering/README.md`. Implementation: `docs/IMPLEMENTATION.md`.

A. Engineering contracts and decision matrices

A.1. Stable exported anchors

Downstream profiles may cite these stable anchors: `V2-LEDGER-01` authoritative internal ledger; `V2-IDENTITY-01` intent/operation/attempt/observation identity; `V2-STATE-01` durable transition and ambiguity; `V2-SIGN-01` contextual signing; `V2-EVM-01` EVM native attempt; `V2-SOL-01` Solana native attempt; `V2-OBS-01` independent observation; `V2-FINALITY-01` four policies; `V2-RECON-01` authoritative reconciliation; `V2-OPS-01` recoverable operations; `V2-TEST-01` evidence pyramid; and `V2-ROADMAP-01` readiness gates. Profiles add thresholds and products; they may not weaken the invariant without a governed exception.

A.2. Adapters and language choices

Table 10. Local and production adapter contract.

Boundary	Local/test	Production
database/messaging	disposable PostgreSQL/broker containers; fault controls	managed/operated HA, PITR, retention, quotas, monitored recovery
signing	fixture local signer and mock HSM with denial/fault modes	dedicated signer plus approved HSM/MPC/custody, workload identity, audit
EVM	Anvil, reviewed test contract, deterministic accounts	selected networks, verified contract/roles, provider diversity, native finality policy
Solana	local validator, fixture mint/program/accounts	selected cluster, reviewed token/program authorities, provider diversity, commitment policy
observation	deterministic poll/stream fixtures and fork injection	independent provider and optional read/verifier/indexer
telemetry/audit	local collector and test evidence sink	controlled OTel pipeline and separately retained audit evidence
policy	versioned fixtures	approved signed bundle, change control, fail-closed distribution

Table 11. Language and SDK evaluation matrix.

Language	Candidate	Good bounded use	Gate before adoption
Java	Web3j / Solana-specific library	primary control plane and adapters	whole-object native vectors, maintenance/security, no SDK types in domain
TypeScript	Viem	independent EVM vectors, tooling, selected edge services	runtime/on-call ownership, strict typing/validation, supply-chain review
Rust	Alloy / native Solana / Anchor	native-performance or program boundary	memory/unsafe review, toolchain reproducibility, operational ownership
Go	go-ethereum packages	node-side tooling or independent EVM verification	API stability, CGO/build profile, service ownership
Python	web3.py	analysis, fixtures, operational tooling	not a hidden production authority; dependency and input controls

A.3. Network, L2, and provider decisions

Network evaluation is per asset and use case. Record native/token support; issuance and administrative authority; transaction/lifetime/signature semantics; blockchain, legal settlement, customer-visible, and accounting finality; reorg/fork and L2 settlement; throughput/latency/fees; RPC/indexer/custody diversity; incident and upgrade governance; regional/compliance fit; reconciliation data; and exit. Ethereum, Base, Solana, Arbitrum, and Optimism receive independent evidence—EVM compatibility does not make their finality or operations identical. An organization L2 additionally evaluates sequencer, batcher/proposer, data availability, proof/dispute, bridge, upgrade, emergency authority, operated nodes, participant access, and shutdown/exit. A managed rollup vendor does not transfer risk by renaming it.

Wallet and custody platforms are candidates to diligence through the same contextual-signing and exit contract. The initial official-document roster is MetaMask Embedded Wallets/Web3Auth, Particle Network, Portal, Privy, Dynamic, Coinbase Developer Platform wallets, Turnkey, Magic, Dfns, Fireblocks, and WalletConnect. (MetaMask 2026; Particle Network 2026; Portal 2026; Privy 2026; Dynamic 2026; Coinbase 2026; Turnkey 2026; Magic 2026; Dfns 2026; Fireblocks 2026; WalletConnect 2026) For each, mark custody/control model, supported native algorithms and exact-message/digest API, independent decoding/policy, key/version/rotation/recovery, approvals, workload/admin identity, audit/export, regions/quotas/SLAs, incident controls, on-chain authority integration, and exit. Official marketing or SDK availability alone establishes none of these. Unsupported, unknown, failed gate, and not evaluated remain explicit.

Build a signing service when the institution needs native semantic policy, provider portability, or controlled evidence that no provider boundary supplies. Buy protected cryptography/custody when certification, key durability, distributed approvals, operations, or legal control is better provided externally. The usual answer is hybrid: own contextual policy, registry, evidence, and provider-neutral ports; buy protected key operations; keep an exercised exit path. Do not build custom cryptography or a proprietary wallet merely for architectural symmetry.

A.4. Implementation status detail

Table 12. Pinned implementation status and limitation.

Capability	Status	Evidence boundary at the pinned commit
domain identities/amounts/lifecycle	implemented	plain Java types, aggregate transitions, and tests; rich states are not runtime progression
authoritative double-entry ledger	planned	no ledger/accounting module or tests; exact money types and operation persistence are not accounting authority
application request acceptance	implemented	durable create/replay/conflict path; no downstream execution
PostgreSQL/Flyway	implemented	operation/request/outbox persistence and migrations; not production operations
Spring API/security	partially implemented	No identity adapter/authentication mechanism is configured; endpoints deny by default; the asset catalog returns empty; tests inject fixture principals. No application role/limit/policy authorization.
outbox	partially implemented	pending row committed; no poller, lease, delivery, inbox, or recovery

Capability	Status	Evidence boundary at the pinned commit
worker lifecycle	planned	no runtime drives operations beyond requested
signer port	interface only	Java port only; no contextual service/provider
chain port	interface only	generic Java port only; no native adapter
EVM execution	planned	no Web3j, contract, Foundry, nonce, submission, or observation path
Solana execution	planned	no SDK/Rust/Anchor, local validator, submission, or observation path
custom Solana program	intentionally excluded	deferred until native token/program gap is proven
observation	interface only	`ChainPort` request/result types and port test; no adapter, independent source, evidence gate, or worker
reconciliation	planned	no comparison service, break workflow, or runtime worker
production deployment/operations	planned	no deploy stack, workload identity, dashboards, DR, or runbooks
provider selection/readiness	unresolved	no provider compatibility or production evidence

A.5. Runbook and failure contract

Every cataloged failure has: stable ID; detection signal; native/business classification; affected operations and evidence query; safe first action; prohibited action; accountable owner; escalation and approval; recovery preconditions; reconciliation and customer/finance impact; closure evidence; and test/drill. Every runbook has a non-production rehearsal date. The machine-readable catalogs in `evidence/` are the canonical companion indexes.

B. Publication contract

The release is portrait US Letter PDF 1.7 with selectable text, embedded fonts, bookmarks, internal links, reader citations, annotated references, and no unresolved references or citations. The canonical PDF and series-level convenience copy are byte-identical. The deterministic source archive contains one versioned root and excludes build output, release output, caches, internal planning files, and repository metadata. Two isolated archive builds must reproduce the canonical PDF byte-for-byte. Visual and accessibility review remains required even after mechanical tests.

C. Annotated sources

The bibliography favors normative internal specifications, standards, protocol specifications, and official project/provider documentation. An annotation states the bounded use and limitation; an access date indicates currency for living documentation. A citation does not imply product approval, production compatibility, or satisfaction of institutional control requirements.

Alloy Contributors (2026). *Alloy Documentation*. URL: https://alloy.rs/ (visited on 2026-07-16)

Categories: primary, rust, evm

Use and limitation: Official Rust EVM toolkit documentation; adoption is bounded by language and operations gates.

Amazon Web Services (2026a). *AWS KMS Sign API*. URL: https://docs.aws.amazon.com/kms/latest/APIReference/API_Sign.html (visited on 2026-07-16)

Categories: primary, hsm, kms, signing
Use and limitation: Official RAW/DIGEST and signing API behavior; it does not independently authorize transaction meaning.

Amazon Web Services (2026b). *CloudHSM PKCS #11 Mechanisms*. URL: https://docs.aws.amazon.com/cloudhsm/latest/userguide/pkcs11-mechanisms.html (visited on 2026-07-16)
Categories: primary, hsm, pkcs11
Use and limitation: Official mechanism support; firmware/client versions and exact encoding need compatibility evidence.

– (2026c). *Key Spec Reference*. URL: https://docs.aws.amazon.com/kms/latest/developerguide/symm-asymm-choose-key-spec.html (visited on 2026-07-16)
Categories: primary, hsm, kms
Use and limitation: Official current algorithm/key support; region, API semantics, quotas, and account policy must be tested.

Anchor Contributors (2026). *Anchor Documentation*. URL: https://www.anchor-lang.com/docs (visited on 2026-07-16)
Categories: primary, solana, rust
Use and limitation: Official framework documentation; generated constraints and upgrade authority still require review.

Anza (2026). *Geyser Plugin Interface*. URL: https://docs.anza.xyz/validator/geyser (visited on 2026-07-16)
Categories: primary, solana, indexing
Use and limitation: Official validator streaming interface; completeness, replay, and failure independence require deployment evidence.

Apache Software Foundation (2026). *The Maven Reactor*. URL: https://maven.apache.org/guides/mini/guide-multiple-modules.html (visited on 2026-07-16)
Categories: primary, java, build
Use and limitation: Official multi-module build behavior; architecture dependency rules require additional enforcement.

Broadcom (2026a). *Spring Boot Actuator Endpoints*. URL: https://docs.spring.io/spring-boot/reference/actuator/endpoints.html (visited on 2026-07-16)
Categories: primary, operations, java
Use and limitation: Official endpoint behavior; exposure and readiness policy require profile controls.

– (2026b). *Spring Boot Reference Documentation*. URL: https://docs.spring.io/spring-boot/reference/ (visited on 2026-07-16)
Categories: primary, java, spring
Use and limitation: Official reference for Spring Boot configuration and production features; implementation remains version-pinned separately.

– (2026c). *Spring Framework Transaction Management*. URL: https://docs.spring.io/spring-framework/reference/data-access/transaction.html (visited on 2026-07-16)
Categories: primary, java, transactions
Use and limitation: Official transaction semantics; does not choose business transaction boundaries.

– (2026d). *Spring Security Reference*. URL: https://docs.spring.io/spring-security/reference/ (visited on 2026-07-16)
Categories: primary, security, java
Use and limitation: Official framework security reference; application authorization and participant scope remain system responsibilities.

Buterin, V. (2015). *EIP-2: Homestead Hard-Fork Changes*. URL: https://eips.ethereum.org/EIPS/eip-2 (visited on 2026-07-16)
Categories: standard, evm, signing

Use and limitation: Normative source for low-s transaction signature validity.

– (2016). *EIP-155: Simple Replay Attack Protection*. URL: https://eips.ethereum.org/EIPS/eip-155 (visited on 2026-07-16)
Categories: standard, evm, signing
Use and limitation: Defines legacy chain-ID replay protection; typed-envelope rules also apply.

Buterin, V. et al. (2019). *EIP-1559: Fee Market Change for ETH 1.0 Chain*. URL: https://eips.ethereum.org/EIPS/eip-1559 (visited on 2026-07-16)
Categories: standard, evm, transactions
Use and limitation: Normative typed-transaction fee fields; network/L2 implementations need separate confirmation.

Caldera (2026). *Caldera Documentation*. URL: https://docs.caldera.xyz/ (visited on 2026-07-16)
Categories: primary, l2, provider
Use and limitation: Official managed-rollup documentation; service claims require contractual and technical verification.

Coinbase (2026). *Developer Platform Non-Custodial Wallets*. URL: https://docs.cdp.coinbase.com/wallets/non-custodial-wallets/overview (visited on 2026-07-16)
Categories: primary, wallet, candidate
Use and limitation: Official candidate documentation; product mode, algorithms, policy, and exit must be evaluated.

Conduit (2026). *Conduit Documentation*. URL: https://docs.conduit.xyz/ (visited on 2026-07-16)
Categories: primary, l2, provider
Use and limitation: Official managed-rollup documentation; no vendor approval or control transfer is inferred.

Dfns (2026). *Dfns Documentation*. URL: https://docs.dfns.co/ (visited on 2026-07-16)
Categories: primary, custody, candidate
Use and limitation: Official candidate documentation; exact native signing and operating model require diligence.

Digital Banking Architecture Program (2026a). *Master Series Specification*. Internal architecture repository
Categories: internal, normative
Use and limitation: Defines series authority and publication boundaries; it is not implementation evidence.

– (2026b). *Volume I Specification*. Internal architecture repository
Categories: internal, normative
Use and limitation: Defines the reference architecture consumed by this engineering companion.

Digital Banking Implementation Repository (2026). *Source and Tests at the Pinned Commit*. Read-only local Git repository
Categories: internal, implementation-evidence
Use and limitation: The exact commit is recorded in the reader guide and evidence ledger; it supports bounded status claims, not production readiness.

Dynamic (2026). *Dynamic Documentation*. URL: https://www.dynamic.xyz/docs (visited on 2026-07-16)
Categories: primary, wallet, candidate
Use and limitation: Official candidate documentation; enterprise control compatibility remains not evaluated.

go-ethereum Authors (2026). *Go API*. URL: https://geth.ethereum.org/docs/developers/dapp-developer/native (visited on 2026-07-16)
Categories: primary, go, evm
Use and limitation: Official Go API guidance; not a recommendation to split the control plane.

Ethereum Foundation (2026a). *Ethereum Transactions*. URL: https://ethereum.org/en/developers/docs/transactions/ (visited on 2026-07-16)
Categories: primary, evm, transactions

Use and limitation: Official developer explanation of transaction fields and signatures; EIPs remain normative for exact encoding.

Ethereum Foundation (2026b). *Gas and Fees*. URL: https://ethereum.org/en/developers/docs/gas/ (visited on 2026-07-16)
Categories: primary, evm, fees
Use and limitation: Official gas concepts; current fee levels and chain-specific policies require live evidence.

Fireblocks (2026). *Fireblocks Developer Documentation*. URL: https://developers.fireblocks.com/ (visited on 2026-07-16)
Categories: primary, custody, candidate
Use and limitation: Official candidate documentation; contract, control, audit, and exit evidence remain required.

Foundry Contributors (2026). *Foundry Book*. URL: https://book.getfoundry.sh/ (visited on 2026-07-16)
Categories: primary, evm, testing
Use and limitation: Official smart-contract toolchain documentation; tool use does not replace independent security review.

Google Cloud (2026). *Cloud KMS Algorithms*. URL: https://cloud.google.com/kms/docs/algorithms (visited on 2026-07-16)
Categories: primary, hsm, kms
Use and limitation: Official algorithm roster; absence or presence is not full native-transaction compatibility.

Josefsson, S. and I. Liusvaara (2017). *RFC 8032: Edwards-Curve Digital Signature Algorithm*. URL: https://www.rfc-editor.org/rfc/rfc8032 (visited on 2026-07-16)
Categories: standard, cryptography, solana
Use and limitation: Normative Ed25519 reference; Solana message serialization and signer APIs remain protocol-specific.

Kubernetes Authors (2026). *Service Accounts*. URL: https://kubernetes.io/docs/concepts/security/service-accounts/ (visited on 2026-07-16)
Categories: primary, identity, kubernetes
Use and limitation: Official service-account behavior; audience, RBAC, token projection, and cluster hardening require configuration evidence.

Magic (2026). *Magic Documentation*. URL: https://docs.magic.link/ (visited on 2026-07-16)
Categories: primary, wallet, candidate
Use and limitation: Official candidate documentation; no fit or security conclusion is asserted.

MetaMask (2026). *Embedded Wallets Documentation*. URL: https://docs.metamask.io/embedded-wallets/ (visited on 2026-07-16)
Categories: primary, wallet, candidate
Use and limitation: Official candidate documentation; custody, signing policy, algorithms, audit, and exit require diligence.

Microsoft (2026). *Azure Key Vault Keys*. URL: https://learn.microsoft.com/en-us/azure/key-vault/keys/about-keys (visited on 2026-07-16)
Categories: primary, hsm, kms
Use and limitation: Official key types and protection boundary; exact curve/API support requires current validation.

OASIS (2023). *PKCS #11 Cryptographic Token Interface*. URL: https://docs.oasis-open.org/pkcs11/pkcs11-spec/v3.1/ (visited on 2026-07-16)
Categories: standard, hsm, cryptography
Use and limitation: Normative token interface; vendor mechanism and lifecycle support vary.

Offchain Labs (2026). *Arbitrum Orbit Documentation*. URL: https://docs.arbitrum.io/launch-orbit-chain/orbit-gentle-introduction (visited on 2026-07-16)

Categories: primary, l2, evm
Use and limitation: Official Orbit introduction; chain configuration, security, data availability, and operations require independent evaluation.

OpenAPI Initiative (2026). *OpenAPI Specification.* URL: https://spec.openapis.org/oas/latest.html (visited on 2026-07-16)
Categories: standard, api
Use and limitation: Normative API-description specification; it does not validate business semantics.

OpenTelemetry Authors (2026). *OpenTelemetry Specification.* URL: https://opentelemetry.io/docs/specs/otel/ (visited on 2026-07-16)
Categories: standard, observability
Use and limitation: Defines telemetry signals and data model; durable audit evidence is a separate control.

OpenZeppelin (2026). *Access Control.* URL: https://docs.openzeppelin.com/contracts/access-control (visited on 2026-07-16)
Categories: primary, evm, security
Use and limitation: Official component documentation; correct role governance and upgrade security remain system obligations.

Optimism Collective (2026). *The OP Stack.* URL: https://docs.optimism.io/op-stack/introduction/op-stack (visited on 2026-07-16)
Categories: primary, l2, evm
Use and limitation: Official stack overview; no production selection or equivalence across OP chains is implied.

Particle Network (2026). *Wallet-as-a-Service Documentation.* URL: https://developers.particle.network/ (visited on 2026-07-16)
Categories: primary, wallet, candidate
Use and limitation: Official candidate documentation; no compatibility or selection claim is made.

Pornin, T. (2013). *RFC 6979: Deterministic Usage of DSA and ECDSA.* URL: https://www.rfc-editor.org/rfc/rfc6979 (visited on 2026-07-16)
Categories: standard, cryptography
Use and limitation: Deterministic nonce-generation reference; managed signer implementations may use controlled alternatives.

Portal (2026). *Portal Documentation.* URL: https://docs.portalhq.io/ (visited on 2026-07-16)
Categories: primary, wallet, candidate
Use and limitation: Official candidate documentation; recovery, policy, audit, and native semantics require testing.

PostgreSQL Global Development Group (2026). *Transaction Isolation.* URL: https://www.postgresql.org/docs/current/transaction-iso.html (visited on 2026-07-16)
Categories: primary, database, concurrency
Use and limitation: Official PostgreSQL isolation behavior; production choices require anomaly and load testing.

Privy (2026). *Privy Documentation.* URL: https://docs.privy.io/ (visited on 2026-07-16)
Categories: primary, wallet, candidate
Use and limitation: Official candidate documentation; no production approval is inferred.

Redgate (2026). *Flyway Documentation.* URL: https://documentation.red-gate.com/flyway (visited on 2026-07-16)
Categories: primary, database, migrations
Use and limitation: Official migration-tool documentation; safe compatibility and rollback remain application design concerns.

Solana Foundation (2026a). *Accounts.* URL: https://solana.com/docs/core/accounts (visited on 2026-07-16)

Categories: primary, solana, accounts
Use and limitation: Official account model; program-specific account invariants remain separate.

Solana Foundation (2026b). *Address Lookup Tables*. URL: https://solana.com/developers/guides/advanced/lookup-tables (visited on 2026-07-16)
Categories: primary, solana, transactions
Use and limitation: Official lookup-table guidance; policy must bind actual table identity and contents.

– (2026c). *isBlockhashValid RPC Method*. URL: https://solana.com/docs/rpc/http/isblockhashvalid (visited on 2026-07-16)
Categories: primary, solana, lifetime
Use and limitation: Official validity query; robust recovery also records the last valid block height and attempt lineage.

– (2026d). *Programs*. URL: https://solana.com/docs/core/programs (visited on 2026-07-16)
Categories: primary, solana, programs
Use and limitation: Official programs, PDAs, and CPI concepts; implementation security requires code-specific review.

– (2026e). *Token Extensions*. URL: https://solana.com/docs/tokens/extensions (visited on 2026-07-16)
Categories: primary, solana, token
Use and limitation: Official Token-2022 extension overview; every enabled extension changes policy and test scope.

– (2026f). *Transaction Fees*. URL: https://solana.com/docs/core/fees (visited on 2026-07-16)
Categories: primary, solana, fees
Use and limitation: Official fee and compute concepts; live levels and provider estimation need current evidence.

– (2026g). *Transactions and Instructions*. URL: https://solana.com/docs/core/transactions (visited on 2026-07-16)
Categories: primary, solana, transactions
Use and limitation: Official transaction/message model; cluster behavior and provider support need test evidence.

– (2026h). *Versioned Transactions*. URL: https://solana.com/developers/guides/advanced/versions (visited on 2026-07-16)
Categories: primary, solana, transactions
Use and limitation: Official versioned-message guidance; SDK and provider version support must be tested.

Solana Program Library Contributors (2026). *Token Program*. URL: https://spl.solana.com/token (visited on 2026-07-16)
Categories: primary, solana, token
Use and limitation: Official SPL Token behavior; deployment authority and exact program/version require profile evidence.

Solidity Authors (2026a). *Contract ABI Specification*. URL: https://docs.soliditylang.org/en/latest/abi-spec.html (visited on 2026-07-16)
Categories: standard, evm, encoding
Use and limitation: Normative ABI encoding reference; contract-specific semantics require reviewed artifacts.

– (2026b). *Security Considerations*. URL: https://docs.soliditylang.org/en/latest/security-considerations.html (visited on 2026-07-16)
Categories: primary, evm, security
Use and limitation: Official Solidity security guidance; not an exhaustive threat model or audit.

SPIFFE Authors (2026). *SPIFFE Specifications*. URL: https://spiffe.io/docs/latest/spiffe-about/spiffe-concepts/ (visited on 2026-07-16)
Categories: standard, identity, security
Use and limitation: Workload identity model; deployment trust domain and authorization remain system-specific.

Standards for Efficient Cryptography Group (2010). *SEC 2: Recommended Elliptic Curve Domain Parameters*. URL: https://www.secg.org/sec2-v2.pdf (visited on 2026-07-16)

Categories: standard, cryptography, evm
Use and limitation: Defines secp256k1 domain parameters; implementation validation and protocol encoding remain separate.

Testcontainers (2026). *Testcontainers for Java.* URL: https://java.testcontainers.org/ (visited on 2026-07-16)
Categories: primary, testing, java
Use and limitation: Official integration-test framework documentation; container fidelity depends on pinned images and configuration.

Turnkey (2026). *Turnkey Documentation.* URL: https://docs.turnkey.com/ (visited on 2026-07-16)
Categories: primary, wallet, candidate
Use and limitation: Official candidate documentation; contextual policy and recovery evidence require validation.

Viem Contributors (2026). *Viem Documentation.* URL: https://viem.sh/ (visited on 2026-07-16)
Categories: primary, typescript, evm
Use and limitation: Official TypeScript EVM library documentation; production adoption requires ownership and compatibility evidence.

Vogelsteller, F. and V. Buterin (2015). *ERC-20: Token Standard.* URL: https://eips.ethereum.org/EIPS/eip-20 (visited on 2026-07-16)
Categories: standard, evm, token
Use and limitation: Defines the token interface and events; mint/burn authority and administration are implementation choices.

WalletConnect (2026). *WalletConnect Documentation.* URL: https://docs.walletconnect.network/ (visited on 2026-07-16)
Categories: primary, wallet, candidate
Use and limitation: Official protocol/platform documentation; it is not itself a custody or signer-policy conclusion.

Web3 Labs (2026). *Web3j Documentation.* URL: https://docs.web3j.io/latest/ (visited on 2026-07-16)
Categories: primary, evm, java
Use and limitation: Official Java EVM library documentation; whole-object and provider compatibility tests remain required.

web3.py Contributors (2026). *web3.py Documentation.* URL: https://web3py.readthedocs.io/ (visited on 2026-07-16)
Categories: primary, python, evm
Use and limitation: Official Python library documentation; tooling use must not become an ungoverned production authority.

D. Glossary

Ambiguous outcome An attempt for which an external effect may have occurred but available evidence cannot yet prove success or failure; it requires inquiry, not blind retry.

Attempt One immutable native execution try linked to an operation, including payload, authority, lifetime, submission, and evidence lineage.

Authoritative internal ledger The institution's system of record for balances, reservations, postings, fees, and governed corrections.

Command canonicalization Versioned deterministic normalization and serialization used to bind idempotency and authorization to exact business meaning.

Compensation A new authorized business effect that offsets an earlier effect where allowed; it never erases history or proves reversal of an irreversible native effect.

Contextual signing Authorization of a decoded native payload against operation, policy, asset, authority, destination, amount, fee/lifetime, and key context before protected signing.

Accounting finality The determination that internal postings and accounting treatment are accepted under ledger policy and further correction requires a governed entry. External reconciliation is evidence and a control input, not the definition.

Blockchain finality Approved native-chain evidence for the exact transaction and state effect under the applicable network/commitment policy, including relevant L2 dependencies.

Customer-visible finality The status and remedy promise exposed to a participant or customer under an approved policy.

Evidence Source-qualified, immutable facts and artifacts that support a bounded claim.

Finality One of four distinct policies: blockchain finality, legal settlement finality, customer-visible finality, or accounting finality.

Inbox/outbox Transactional persistence patterns for deduplicated consumption and reliable publication; they support exactly-once business effects over at-least-once delivery.

Legal settlement finality Finality established by applicable law, rules, agreements, and legal analysis; not inferred solely from technical confirmation.

Observation A source-qualified native fact about submission, inclusion, execution, canonical lineage, or finality.

Operation The durable business workflow created from an accepted payment intent.

PDA Program-derived address: a Solana address deterministically derived for a program and authorized through that program, not an ordinary private-key account.

Reconciliation Comparison of authoritative ledger/business intent with operation, signer, submission, and independent native evidence, producing either agreement or an owned break.

Recovery Durable inquiry and authorized progression of stalled or ambiguous work after a process, dependency, provider, or regional failure.

SignerPort The application-facing contextual authorization/signing capability; never a generic arbitrary signing oracle.

Transition An immutable, expected-version state change with cause, actor, policy, time, and evidence links.

Workload identity Short-lived, audience-bound service identity used to authorize runtime access without static embedded credentials.